

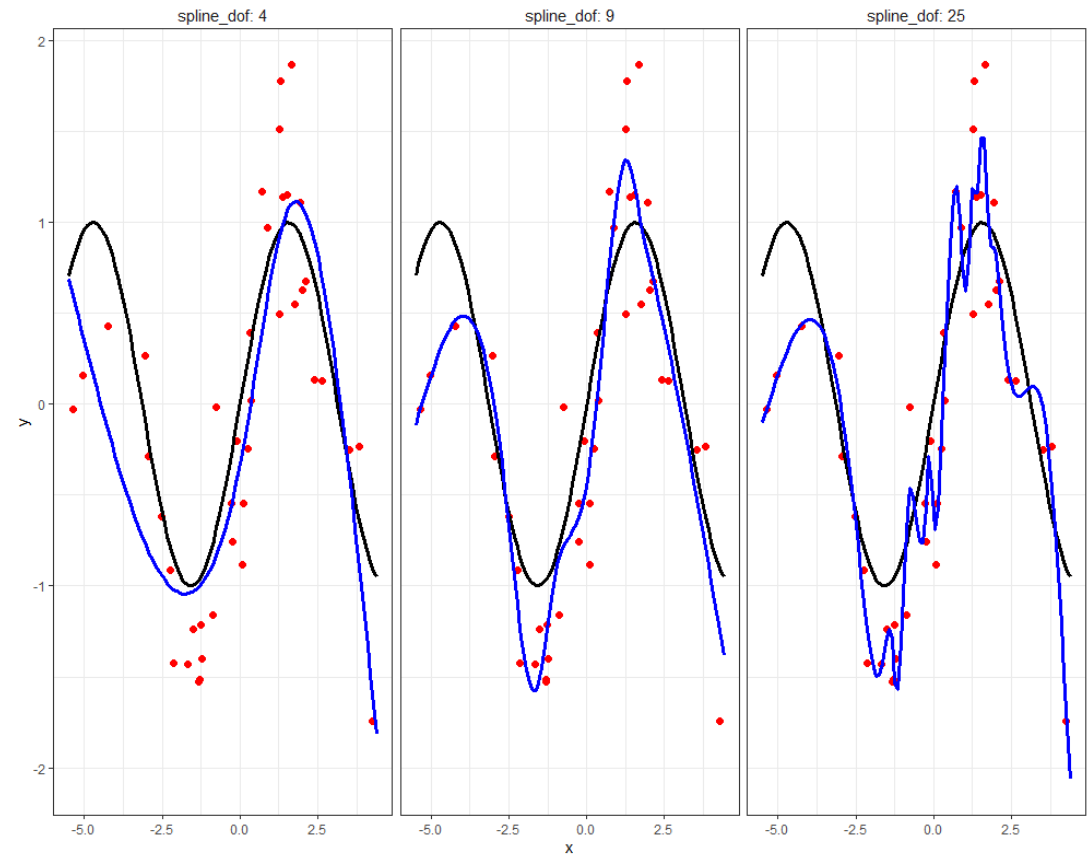
CS1675/2075: Introduction to Machine Learning

Week 10: Model Complexity and Information Criteria

Fall 2024

Let's continue fitting the NOISY sine wave with natural splines

- We previously focused on FEATURE generation.
- We saw that spline FEATURES are derived from the input.
- We fit 3 different degrees-of-freedom natural spline models.
- Visually examined their performance through predictions.

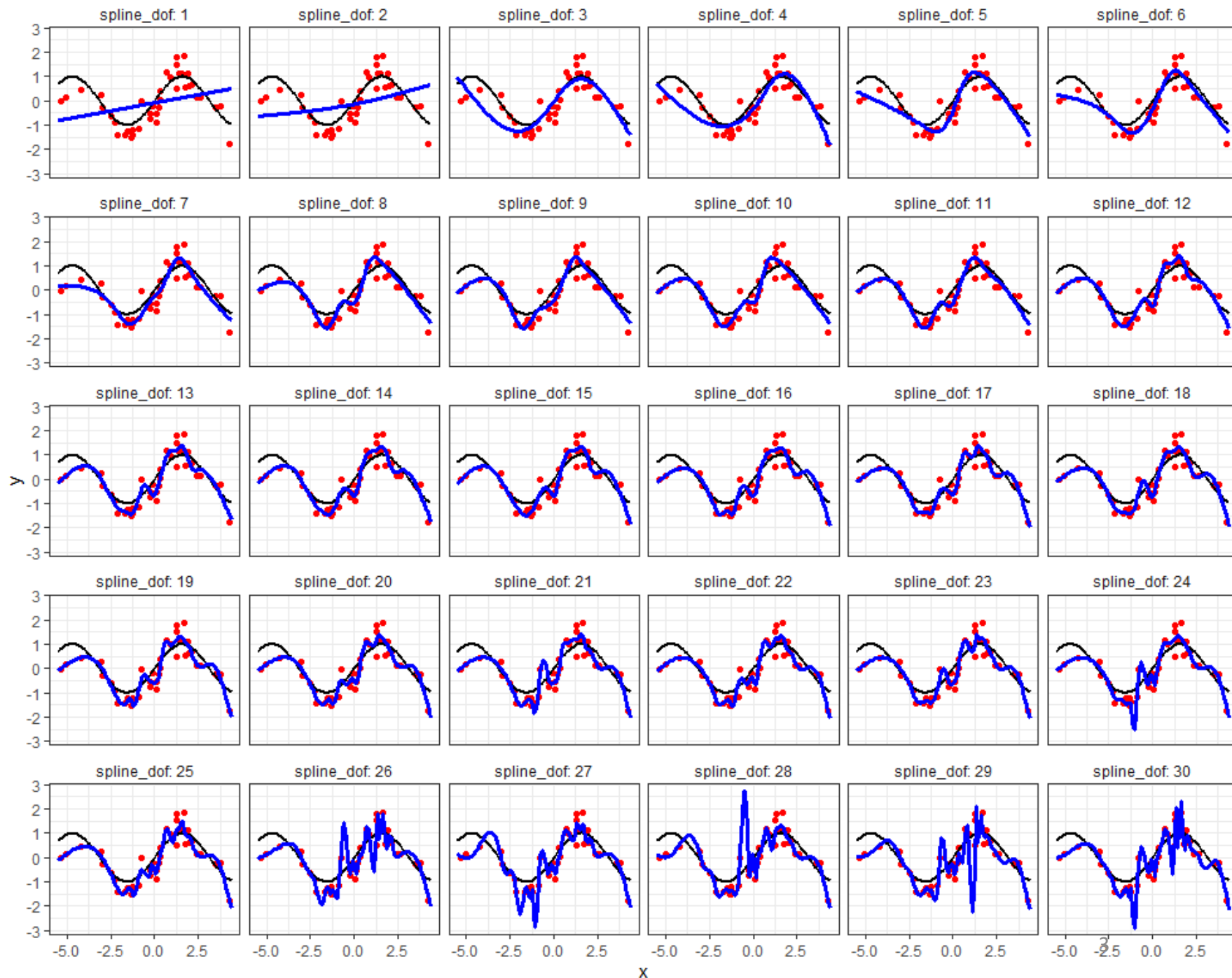


However, nothing is stopping us from trying out MANY degrees-of-freedom!

What if we tried 30 natural spline models!?!

The figure to the right shows the predictions from a 1 degree-of-freedom natural spline up to and including a 30 degree-of-freedom natural spline!!

WHICH MODEL IS THE BEST?



Model selection

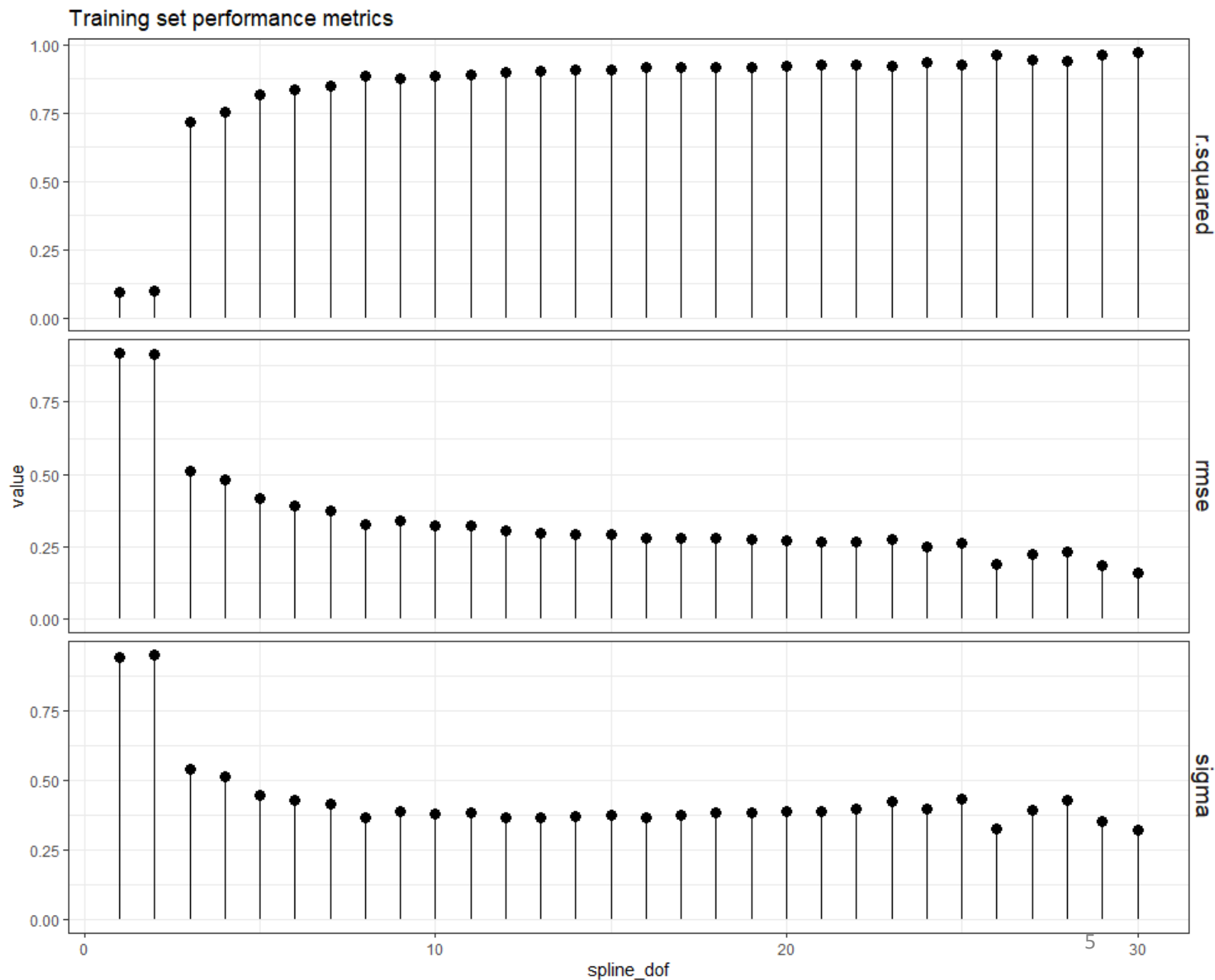
- We considered model selection from a non-Bayesian perspective at the beginning of the semester.
- Back then we saw that more complex models will perform better on the training set!
- The degree-of-freedom controls complexity of linear basis function models like natural splines.
- Higher degree-of-freedom the more complex the model is!

Just as we saw at the beginning of the semester...higher complexity is associated with BETTER training set performance!

Model with the highest TRAINING set R-squared – most complex model!!!

Model with the lowest TRAINING set RMSE – most complex model!!!

If we use training set performance metrics alone...we will NOT identify the model that will do the best on NEW data!!!!



But...the previous performance metrics were from non-Bayesian models...

- We used $\text{lm}()$ to fit the models at the beginning of the semester.
- We now know that $\text{lm}()$ estimates the regression coefficients, β , and residual error, σ , by maximizing the likelihood.
- Thus, fitting models by maximizing the likelihood can lead to OVERFITTING!!!!

Does a Bayesian model also OVERFIT the training data????

Bayesian linear basis function models

- Our goal is to learn the posterior distribution on the unknown $\boldsymbol{\beta}$ -parameters and the unknown noise σ .
- Assuming the β -parameters are a-priori independent of the noise the posterior is:

$$p(\boldsymbol{\beta}, \sigma \mid \boldsymbol{\Phi}, \mathbf{y}) \propto p(\mathbf{y} \mid \boldsymbol{\Phi}, \boldsymbol{\beta}, \sigma) p(\boldsymbol{\beta}) p(\sigma)$$

The likelihood can still be factored because of conditional independence!

$$p(\boldsymbol{\beta}, \sigma \mid \boldsymbol{\Phi}, y) \propto \prod_{n=1}^N \left(p(y_n \mid \boldsymbol{\Phi}_{n,:}, \boldsymbol{\beta}, \sigma) \right) p(\boldsymbol{\beta}) p(\sigma)$$

The likelihood can still be factored because of conditional independence!

$$p(\boldsymbol{\beta}, \sigma \mid \boldsymbol{\Phi}, y) \propto \prod_{n=1}^N \left(p(y_n \mid \boldsymbol{\phi}_{n,:}, \boldsymbol{\beta}, \sigma) \right) p(\boldsymbol{\beta}) p(\sigma)$$

What distribution type is the likelihood for a linear model?

The likelihood can still be factored because of conditional independence!

$$p(\boldsymbol{\beta}, \sigma \mid \boldsymbol{\Phi}, y) \propto \prod_{n=1}^N \left(\text{normal}(y_n \mid \boldsymbol{\phi}_{n,:}\boldsymbol{\beta}, \sigma) \right) p(\boldsymbol{\beta})p(\sigma)$$

GAUSSIAN!!

The mean of the n -th output observation is:

$$\boldsymbol{\phi}_{n,:}\boldsymbol{\beta}$$

Common prior specification on the unknown coefficients

- Although it is not required, it is common to assume all the coefficients are independent a-priori.
- It is also common to assume a shared prior mean, μ_β , and shared prior standard deviation, τ_β .

$$p(\boldsymbol{\beta}) = \prod_{j=0}^J \left(p(\beta_j) \right) = \prod_{j=0}^J \left(\text{normal}(\beta_j \mid \mu_\beta, \tau_\beta) \right)$$

Usually $\mu_\beta = 0$, but it is NOT required!

Complete Bayesian linear basis function model with common prior specification

$$y_n \mid \mu_n, \sigma \sim \text{normal}(y_n \mid \mu_n, \sigma)$$

$$\mu_n = \boldsymbol{\phi}_{n,:} \boldsymbol{\beta}$$

$$\beta_j \mid \mu_\beta, \tau_\beta \sim \prod_{j=0}^J \left(\text{normal}(\beta_j \mid \mu_\beta, \tau_\beta) \right)$$

$$\sigma \mid \lambda \sim \text{Exp}(\sigma \mid \lambda)$$

Another option is to use a separate prior standard deviation for the intercept, β_0

- The intercept is given its own set of prior hyperparameters: μ_0, τ_0
- The basis feature slopes, β_j , use common hyperparameters: μ_β, τ_β

$$p(\boldsymbol{\beta}) = \text{normal}(\beta_0 \mid \mu_0, \tau_0) \times \prod_{j=1}^J \left(\text{normal}(\beta_j \mid \mu_\beta, \tau_\beta) \right)$$

How many unknowns do we need to learn?

- Depends on the degree of the basis function!
- If the basis function degree is J , we have $J + 1$ unknown regression coefficients.
- But...we have a total of $J + 2$ unknowns! Why?

We will still need to apply a transformation to σ to perform the Laplace Approximation

- Let's use the log-transform, as you have used in your assignments:

$$\varphi = \log[\sigma]$$

- The log-posterior function accounting for the change-of-variables is similar to that for the unknown constant mean and “simple” linear regression!

Bayesian linear model log-posterior is VERY SIMILAR to the log-posterior associated with the unknown constant average!!!!

```
lm_logpost <- function(unknowns, my_info)
{
  # specify the number of unknown beta parameters
  length_beta <-

  # extract the beta parameters from the `unknowns` vector
  beta_v <-

  # extract the unbounded noise parameter, varphi
  lik_varphi <-

  # back-transform from varphi to sigma
  lik_sigma <-

  # extract design matrix
  X <-

  # calculate the linear predictor
  mu <-

  # evaluate the log-likelihood
  log_lik <-

  # evaluate the log-prior
  log_prior_beta <-

  log_prior_sigma <-

  # add the mean trend prior and noise prior together
  log_prior <-

  # account for the transformation
  log_derive_adjust <-

  # sum together

}
```

HW07 log-posterior function

```
my_cv_logpost <- function(unknowns, my_info)
{
  # unpack the unknowns into separate variables
  lik_mu <- unknowns[1]
  lik_varphi <- unknowns[2]

  # back transform to sigma
  lik_sigma <- exp(lik_varphi)

  # calculate the log-likelihood
  log_lik <- sum(dnorm(x = my_info$xobs,
                      mean = lik_mu,
                      sd = lik_sigma,
                      log = TRUE))

  # calculate the log-prior on mu
  log_prior_mu <- dnorm(x = lik_mu,
                      mean = my_info$mu_0,
                      sd = my_info$tau_0,
                      log = TRUE)

  # calculate the log-prior on sigma
  log_prior_sigma <- dexp(x = lik_sigma,
                      rate = my_info$sigma_rate,
                      log = TRUE)

  # calculate the log-derivative adjustment
  log_deriv_adjust <- lik_varphi

  # return the (un-normalized) log-posterior
  log_lik + log_prior_mu + log_prior_sigma + log_deriv_adjust
}
```

HW06 log-posterior function

Bayesian linear model log-posterior is **VERY SIMILAR** to the log-posterior associated with the unknown constant average!!!!

```
lm_logpost <- function(unknowns, my_info)
{
  # specify the number of unknown beta parameters
  length_beta <-

  # extract the beta parameters from the `unknowns` vector
  beta_v <-

  # extract the unbounded noise parameter, varphi
  lik_varphi <-

  # back-transform from varphi to sigma
  lik_sigma <-

  # extract design matrix
  X <-

  # calculate the linear predictor
  mu <-

  # evaluate the log-likelihood
  log_lik <-

  # evaluate the log-prior
  log_prior_beta <-

  log_prior_sigma <-

  # add the mean trend prior and noise prior together
  log_prior <-

  # account for the transformation
  log_derive_adjust <-

  # sum together

}
```

HW07 log-posterior function

Both involve conditional independent Gaussian log-likelihoods.

The likelihood is where the observed output is **compared** to the mean output.

```
my_cv_logpost <- function(unknowns, my_info)
{
  # unpack the unknowns into separate variables
  lik_mu <- unknowns[1]
  lik_varphi <- unknowns[2]

  # back transform to sigma
  lik_sigma <- exp(lik_varphi)

  # calculate the log-likelihood
  log_lik <- sum(dnorm(x = my_info$xobs,
                      mean = lik_mu,
                      sd = lik_sigma,
                      log = TRUE))

  # calculate the log-prior on mu
  log_prior_mu <- dnorm(x = lik_mu,
                      mean = my_info$mu_0,
                      sd = my_info$tau_0,
                      log = TRUE)

  # calculate the log-prior on sigma
  log_prior_sigma <- dexp(x = lik_sigma,
                      rate = my_info$sigma_rate,
                      log = TRUE)

  # calculate the log-derivative adjustment
  log_deriv_adjust <- lik_varphi

  # return the (un-normalized) log-posterior
  log_lik + log_prior_mu + log_prior_sigma + log_deriv_adjust
}
```

HW06 log-posterior function

Bayesian linear model log-posterior is **VERY SIMILAR** to the log-posterior associated with the unknown constant average!!!!

```
lm_logpost <- function(unknowns, my_info)
{
  # specify the number of unknown beta parameters
  length_beta <-

  # extract the beta parameters from the `unknowns` vector
  beta_v <-

  # extract the unbounded noise parameter, varphi
  lik_varphi <-

  # back-transform from varphi to sigma
  lik_sigma <-

  # extract design matrix
  X <-

  # calculate the linear predictor
  mu <-

  # evaluate the log-likelihood
  log_lik <-

  # evaluate the log-prior
  log_prior_beta <-

  log_prior_sigma <-

  # add the mean trend prior and noise prior together
  log_prior <-

  # account for the transformation
  log_derive_adjust <-

  # sum together

}
```

HW07 log-posterior function

Both involve conditional independent Gaussian log-likelihoods.

The likelihood standard deviation, noise, or residual error is constant!!!

```
my_cv_logpost <- function(unknowns, my_info)
{
  # unpack the unknowns into separate variables
  lik_mu <- unknowns[1]
  lik_varphi <- unknowns[2]

  # back transform to sigma
  lik_sigma <- exp(lik_varphi)

  # calculate the log-likelihood
  log_lik <- sum(dnorm(x = my_info$xobs,
                      mean = lik_mu,
                      sd = lik_sigma,
                      log = TRUE))

  # calculate the log-prior on mu
  log_prior_mu <- dnorm(x = lik_mu,
                      mean = my_info$mu_0,
                      sd = my_info$tau_0,
                      log = TRUE)

  # calculate the log-prior on sigma
  log_prior_sigma <- dexp(x = lik_sigma,
                      rate = my_info$sigma_rate,
                      log = TRUE)

  # calculate the log-derivative adjustment
  log_deriv_adjust <- lik_varphi

  # return the (un-normalized) log-posterior
  log_lik + log_prior_mu + log_prior_sigma + log_deriv_adjust
}
```

HW06 log-posterior function

Bayesian linear model log-posterior is **VERY SIMILAR** to the log-posterior associated with the unknown constant average!!!!

```
lm_logpost <- function(unknowns, my_info)
{
  # specify the number of unknown beta parameters
  length_beta <-

  # extract the beta parameters from the `unknowns` vector
  beta_v <-

  # extract the unbounded noise parameter, varphi
  lik_varphi <-

  # back-transform from varphi to sigma
  lik_sigma <-

  # extract design matrix
  X <-

  # calculate the linear predictor
  mu <-

  # evaluate the log-likelihood
  log_lik <-

  # evaluate the log-prior
  log_prior_beta <-

  log_prior_sigma <-

  # add the mean trend prior and noise prior together
  log_prior <-

  # account for the transformation
  log_derive_adjust <-

  # sum together

}
```

HW07 log-posterior function

Both use Priors to constrain the unknown parameters.

Both assume ALL parameters are a-priori independent!!!!

```
my_cv_logpost <- function(unknowns, my_info)
{
  # unpack the unknowns into separate variables
  lik_mu <- unknowns[1]
  lik_varphi <- unknowns[2]

  # back transform to sigma
  lik_sigma <- exp(lik_varphi)

  # calculate the log-likelihood
  log_lik <- sum(dnorm(x = my_info$xobs,
                      mean = lik_mu,
                      sd = lik_sigma,
                      log = TRUE))

  # calculate the log-prior on mu
  log_prior_mu <- dnorm(x = lik_mu,
                      mean = my_info$mu_0,
                      sd = my_info$tau_0,
                      log = TRUE)

  # calculate the log-prior on sigma
  log_prior_sigma <- dexp(x = lik_sigma,
                      rate = my_info$sigma_rate,
                      log = TRUE)

  # calculate the log-derivative adjustment
  log_derive_adjust <- lik_varphi

  # return the (un-normalized) log posterior
  log_lik <- log_prior_mu + log_prior_sigma + log_derive_adjust
}
```

HW06 log-posterior function

Bayesian linear model log-posterior is **VERY SIMILAR** to the log-posterior associated with the unknown constant average!!!!

```
lm_logpost <- function(unknowns, my_info)
{
  # specify the number of unknown beta parameters
  length_beta <-

  # extract the beta parameters from the `unknowns` vector
  beta_v <-

  # extract the unbounded noise parameter, varphi
  lik_varphi <-

  # back-transform from varphi to sigma
  lik_sigma <-

  # extract design matrix
  X <-

  # calculate the linear predictor
  mu <-

  # evaluate the log-likelihood
  log_lik <-

  # evaluate the log-prior
  log_prior_beta <-

  log_prior_sigma <-

  # add the mean trend prior and noise prior together
  log_prior <-

  # account for the transformation
  log_deriv_adjust <-

  # sum together
}
```

HW07 log-posterior function

Both assume a log-transformation is applied to σ .

Thus, both formulations are NOT learning σ directly. Instead, both formulations learn the log-transformation, $\varphi = \log(\sigma)$.

The derivative adjustment is accounted for in BOTH log-posteriors.

```
my_cv_logpost <- function(unknowns, my_info)
{
  # unpack the unknowns into separate variables
  lik_mu <- unknowns[1]
  lik_varphi <- unknowns[2]

  # back transform to sigma
  lik_sigma <- exp(lik_varphi)

  # calculate the log-likelihood
  log_lik <- sum(dnorm(x = my_info$xobs,
                      mean = lik_mu,
                      sd = lik_sigma,
                      log = TRUE))

  # calculate the log-prior on mu
  log_prior_mu <- dnorm(x = lik_mu,
                      mean = my_info$mu_0,
                      sd = my_info$tau_0,
                      log = TRUE)

  # calculate the log-prior on sigma
  log_prior_sigma <- dexp(x = lik_sigma,
                      rate = my_info$sigma_rate,
                      log = TRUE)

  # calculate the log-derivative adjustment
  log_deriv_adjust <- lik_varphi

  # return the (un-normalized) log-posterior
  log_lik + log_prior_mu + log_prior_sigma + log_deriv_adjust
}
```

HW06 log-posterior function

But, there are a few important differences in implementation!!!!!!

```
lm_logpost <- function(unknowns, my_info)
{
  # specify the number of unknown beta parameters
  length_beta <-

  # extract the beta parameters from the `unknowns` vector
  beta_v <-

  # extract the unbounded noise parameter, varphi
  lik_varphi <-

  # back-transform from varphi to sigma
  lik_sigma <-

  # extract design matrix
  X <-

  # calculate the linear predictor
  mu <-

  # evaluate the log-Likelihood
  log_lik <-

  # evaluate the log-prior
  log_prior_beta <-

  log_prior_sigma <-

  # add the mean trend prior and noise prior together
  log_prior <-

  # account for the transformation
  log_derive_adjust <-

  # sum together

}
```

HW07 log-posterior function

The unknown constant average assumed a CONSTANT mean output!!!

The linear model requires a matrix operation to calculate the mean output!!!

$$\mu = X\beta \text{ or } \mu = \Phi\beta$$

The mean output depends on the features derived from the input!!

```
my_cv_logpost <- function(unknowns, my_info)
{
  # separate the unknowns into separate variables
  lik_mu <- unknowns[1]
  lik_varphi <- unknowns[2]

  # back transform to sigma
  lik_sigma <- exp(lik_varphi)

  # calculate the log-Likelihood
  log_lik <- sum(dnorm(x = my_info$xobs,
                      mean = lik_mu,
                      sd = lik_sigma,
                      log = TRUE))

  # calculate the log-prior on mu
  log_prior_mu <- dnorm(x = lik_mu,
                      mean = my_info$mu_0,
                      sd = my_info$tau_0,
                      log = TRUE)

  # calculate the log-prior on sigma
  log_prior_sigma <- dexp(x = lik_sigma,
                      rate = my_info$sigma_rate,
                      log = TRUE)

  # calculate the log-derivative adjustment
  log_deriv_adjust <- lik_varphi

  # return the (un-normalized) log-posterior
  log_lik + log_prior_mu + log_prior_sigma + log_deriv_adjust
}
```

HW06 log-posterior function

But, there are a few important differences in implementation!!!!!!

```
lm_logpost <- function(unknowns, my_info)
{
  # specify the number of unknown beta parameters
  length_beta <-

  # extract the beta parameters from the `unknowns` vector
  beta_v <-

  # extract the unbounded noise parameter, varphi
  lik_varphi <-

  # back-transform from varphi to sigma
  lik_sigma <-

  # extract design matrix
  X <-

  # calculate the linear predictor
  mu <-

  # evaluate the log-likelihood
  log_lik <-

  # evaluate the log-prior
  log_prior_beta <-

  log_prior_sigma <-

  # add the mean trend prior and noise prior together
  log_prior <-

  # account for the transformation
  log_deriv_adjust <-

  # sum together

}
```

HW07 log-posterior function

There are as many unknown regression coefficients, β , as there are **COLUMNS** of the design matrix, $\mathbf{X}$ or Φ .

We are assuming all regression coefficients are a-priori independent Gaussians!!!

How can you program the PRODUCT of independent Gaussians in the LOG-space???

```
my_cv_logpost <- function(unknowns, my_info)
{
  # separate the unknowns into separate variables
  lik_mu <- unknowns[1]
  lik_varphi <- unknowns[2]

  # back transform to sigma
  lik_sigma <- exp(lik_varphi)

  # calculate the log-likelihood
  log_lik <- sum(dnorm(x = my_info$xobs,
                      mean = lik_mu,
                      sd = lik_sigma,
                      log = TRUE))

  # calculate the log-prior on mu
  log_prior_mu <- dnorm(x = lik_mu,
                       mean = my_info$mu_0,
                       sd = my_info$tau_0,
                       log = TRUE)

  # calculate the log-prior on sigma
  log_prior_sigma <- dexp(x = lik_sigma,
                        rate = my_info$sigma_rate,
                        log = TRUE)

  # calculate the log-derivative adjustment
  log_deriv_adjust <- lik_varphi

  # return the (un-normalized) log-posterior
  log_lik + log_prior_mu + log_prior_sigma + log_deriv_adjust
}
```

HW06 log-posterior function

Include the `my_laplace()` function which performs the optimization and covariance matrix calculation

```
my_laplace <- function(start_guess, logpost_func, ...)  
{  
  # code adapted from the `LearnBayes` function `laplace()`  
  fit <- optim(start_guess,  
              logpost_func,  
              gr = NULL,  
              ...,  
              method = "BFGS",  
              hessian = TRUE,  
              control = list(fnscale = -1, maxit = 1001))  
  
  mode <- fit$par  
  h <- -solve(fit$hessian)  
  p <- length(mode)  
  int <- p/2 * log(2 * pi) + 0.5 * log(det(h)) + logpost_func(mode, ...)  
  list(mode = mode,  
        var_matrix = h,  
        log_evidence = int,  
        converge = ifelse(fit$convergence == 0,  
                           "YES",  
                           "NO"),  
        iter_counts = fit$counts[1])  
}
```


Execute the Laplace Approximation...does the initial guess matter?

- NO!!! Still a linear model!
- There is one unique maximum!

Laplace Approximation applied to the 4 degrees-of-freedom spline model

- Create the list of required information.
- Pass in the observations and the design matrix.
- Specify the prior parameters.

```
Xmat4 <- model.matrix(y ~ splines::ns(x, 4), data = demo_sine)

info_spline_4 <- list(
  yobs = demo_sine$y,
  design_matrix = Xmat4,
  mu_beta = 0,
  tau_beta = 5,
  sigma_rate = 1
)
```

We will discuss why a prior standard deviation, τ_β , of 5 is used later.

Execute the Laplace Approximation on the 4 degree-of-freedom natural spline

```
> laplace_spline_4 <- my_laplace(rep(0, ncol(Xmat4) + 1), lm_logpost, info_spline_4)
>
> laplace_spline_4$converge
[1] "YES"
>
> laplace_spline_4$mode
[1] 0.5584947 -1.0244500 2.3613779 -2.5077069 -1.0182893 -0.7291431
>
> laplace_spline_4$var_matrix
      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
[1,] 0.1049486918 -0.0915700825 -0.0631076010 -0.232010850 -0.0254923963 -0.0006049006
[2,] -0.0915700825 0.1362179081 0.0308247080 0.161676336 0.0567930272 0.0005738298
[3,] -0.0631076010 0.0308247080 0.1109573507 0.142404569 -0.0191822664 0.0001412405
[4,] -0.2320108499 0.1616763361 0.1424045690 0.619663403 0.0619896001 0.0015552516
[5,] -0.0254923963 0.0567930272 -0.0191822664 0.061989600 0.1539873740 0.0004166962
[6,] -0.0006049006 0.0005738298 0.0001412405 0.001555252 0.0004166962 0.0122822522
>
> sqrt( diag(laplace_spline_4$var_matrix) )
[1] 0.3239579 0.3690771 0.3331026 0.7871870 0.3924123 0.1108253
>
> cov2cor(laplace_spline_4$var_matrix)
      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
[1,] 1.00000000 -0.76585753 -0.584810440 -0.90979146 -0.200530109 -0.016848316
[2,] -0.76585753 1.00000000 0.250728573 0.55648240 0.392134775 0.014029009
[3,] -0.58481044 0.25072857 1.000000000 0.54308519 -0.146750410 0.003825976
[4,] -0.90979146 0.55648240 0.543085191 1.00000000 0.200677349 0.017827224
[5,] -0.20053011 0.39213478 -0.146750410 0.20067735 1.000000000 0.009581599
[6,] -0.01684832 0.01402901 0.003825976 0.01782722 0.009581599 1.000000000
```

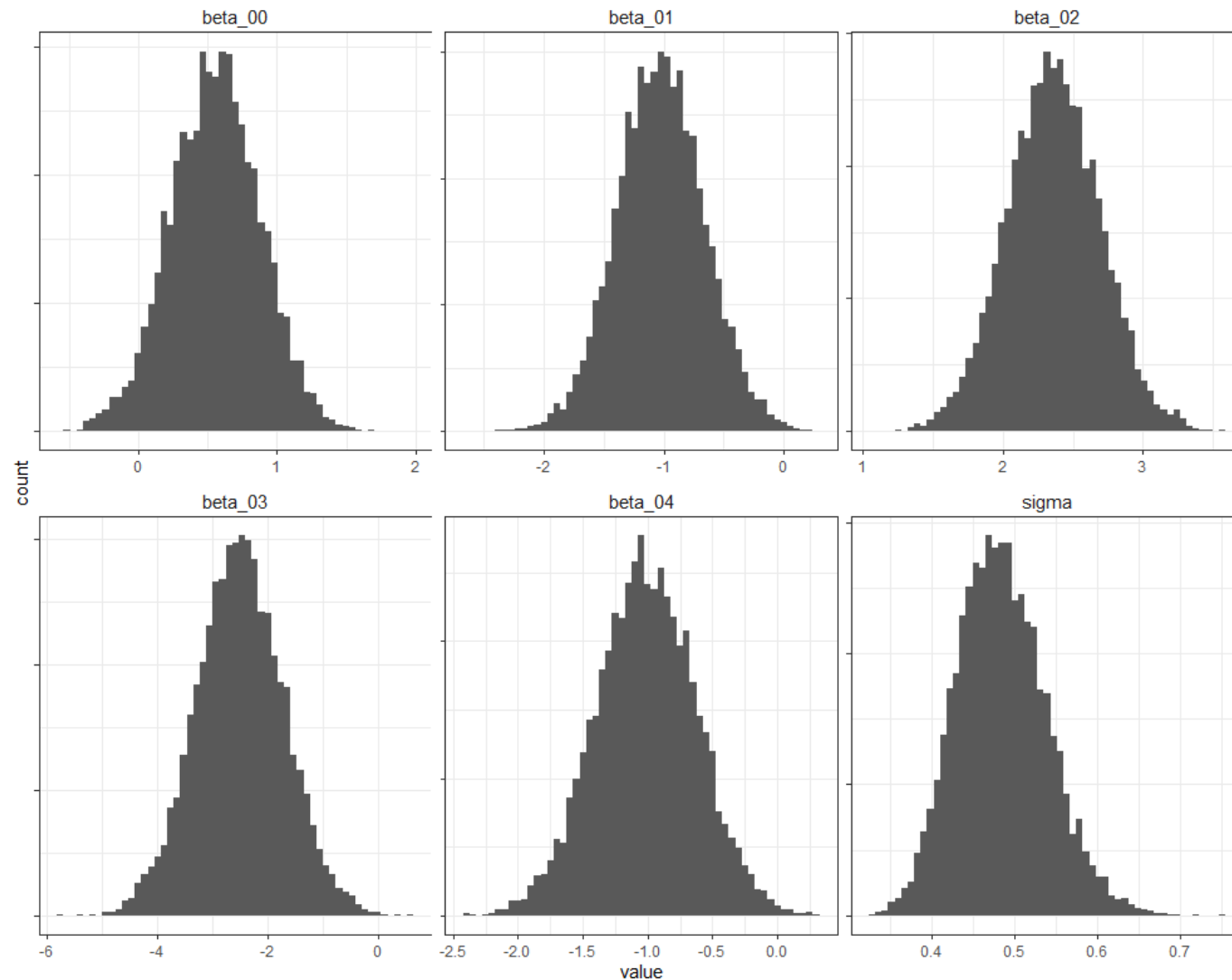
Generate posterior samples to help explore the results

```
generate_post_samples <- function(mvn_result, length_beta, num_samples)
{
  MASS::mvrnorm(n = num_samples,
                mu = mvn_result$mode,
                Sigma = mvn_result$var_matrix) %>%
  as.data.frame() %>% tibble::as_tibble() %>%
  purrr::set_names(c(sprintf("beta_%02d", (1:length_beta) - 1), "phi")) %>%
  mutate(sigma = exp(phi))
}
```

Specifically for the 4 degrees-of-freedom spline

```
set.seed(12312)
post_samples_4 <- generate_post_samples(laplace_spline_4, ncol(Xmat4), 1e4)
```

Posterior histograms of the parameters in the 4 degrees-of-freedom spline model



Apply the Laplace Approximation to 9 degrees-of-freedom

- Create the list of required information again.

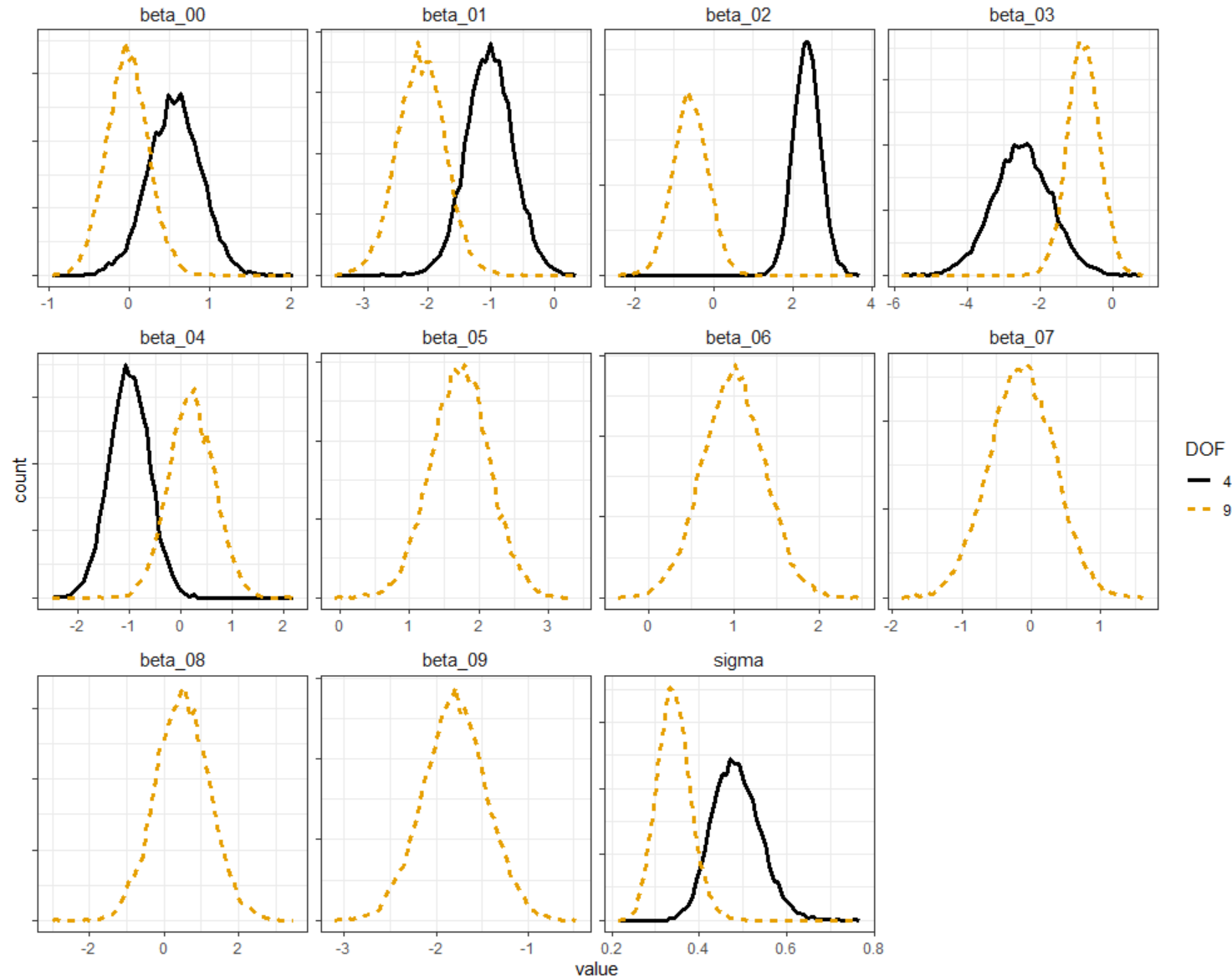
```
Xmat9 <- model.matrix(y ~ splines::ns(x, 9), data = demo_sine)

info_spline_9 <- list(
  yobs = demo_sine$y,
  design_matrix = Xmat9,
  mu_beta = 0,
  tau_beta = 5,
  sigma_rate = 1
)
```

Execute the Laplace Approximation

```
> laplace_spline_9 <- my_laplace(rep(0, ncol(Xmat9) + 1), lm_logpost, info_spline_9)
>
> laplace_spline_9$converge
[1] "YES"
>
> laplace_spline_9$mode
[1] -0.02854112 -2.10145557 -0.59035887 -0.81532608  0.20085372  1.72981337  0.98493615 -0.13566510  0.48916112 -1.79733662 -1.07827325
>
> dim(laplace_spline_9$var_matrix)
[1] 11 11
>
> sqrt(diag(laplace_spline_9$var_matrix))
[1] 0.2541114 0.3711742 0.4418448 0.4333291 0.4416853 0.4210573 0.3646475 0.4685878 0.7223403 0.3456081 0.1111043
```

Compare the posterior histograms of the parameters in the two different spline models



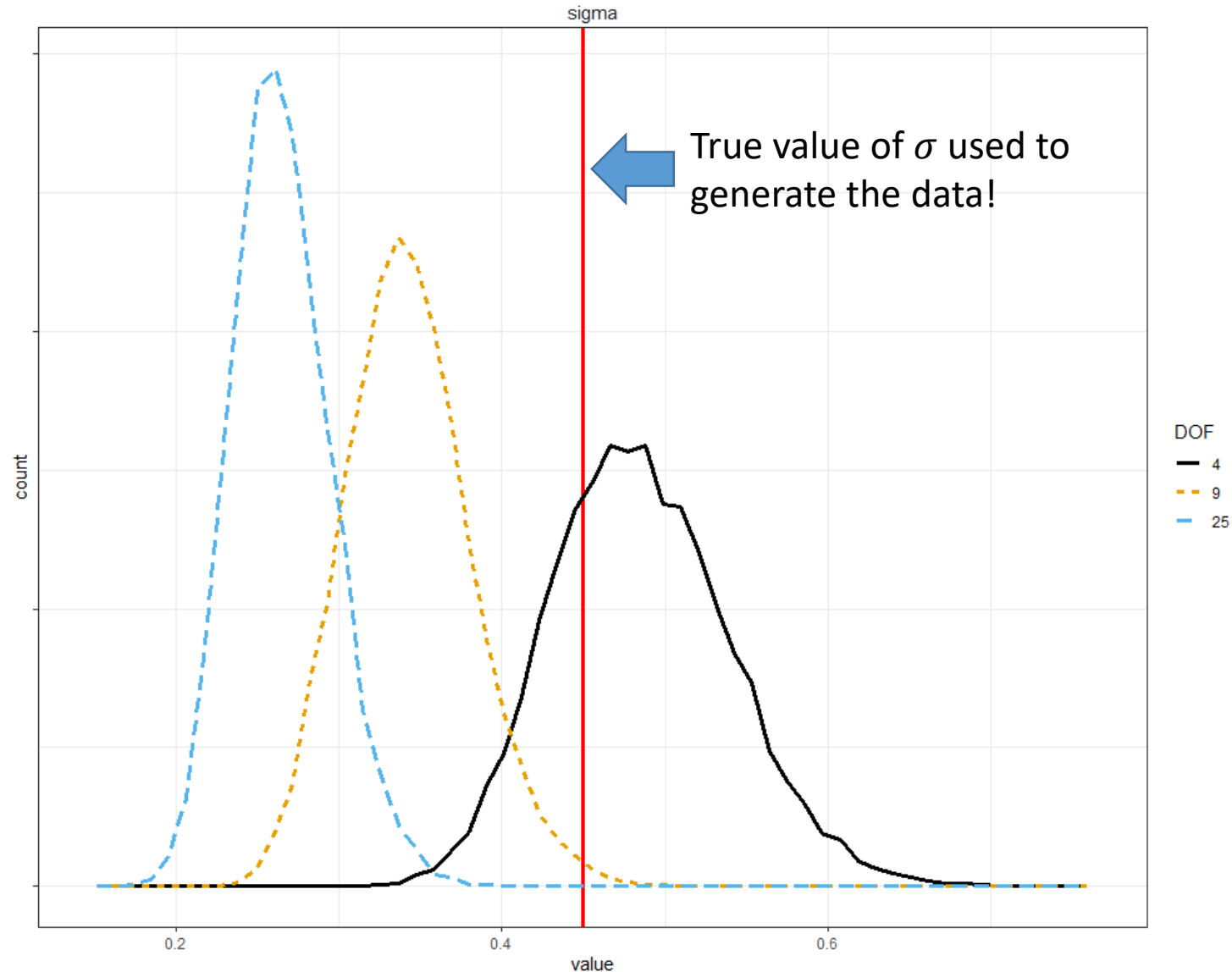
Execute the Laplace Approximation on the 25 degree-of-freedom natural spline!

```
> Xmat25 <- model.matrix(y ~ splines::ns(x, 25), data = demo_sine)
>
> Xmat25 %>% dim()
[1] 41 26
>
> info_spline_25 <- list(
+   yobs = demo_sine$y,
+   design_matrix = Xmat25,
+   mu_beta = 0,
+   tau_beta = 5,
+   sigma_rate = 1
+ )
>
> laplace_spline_25 <- my_laplace(rep(0, ncol(Xmat25) + 1), lm_logpost, info_spline_25)
>
> laplace_spline_25$converge
[1] "YES"
```

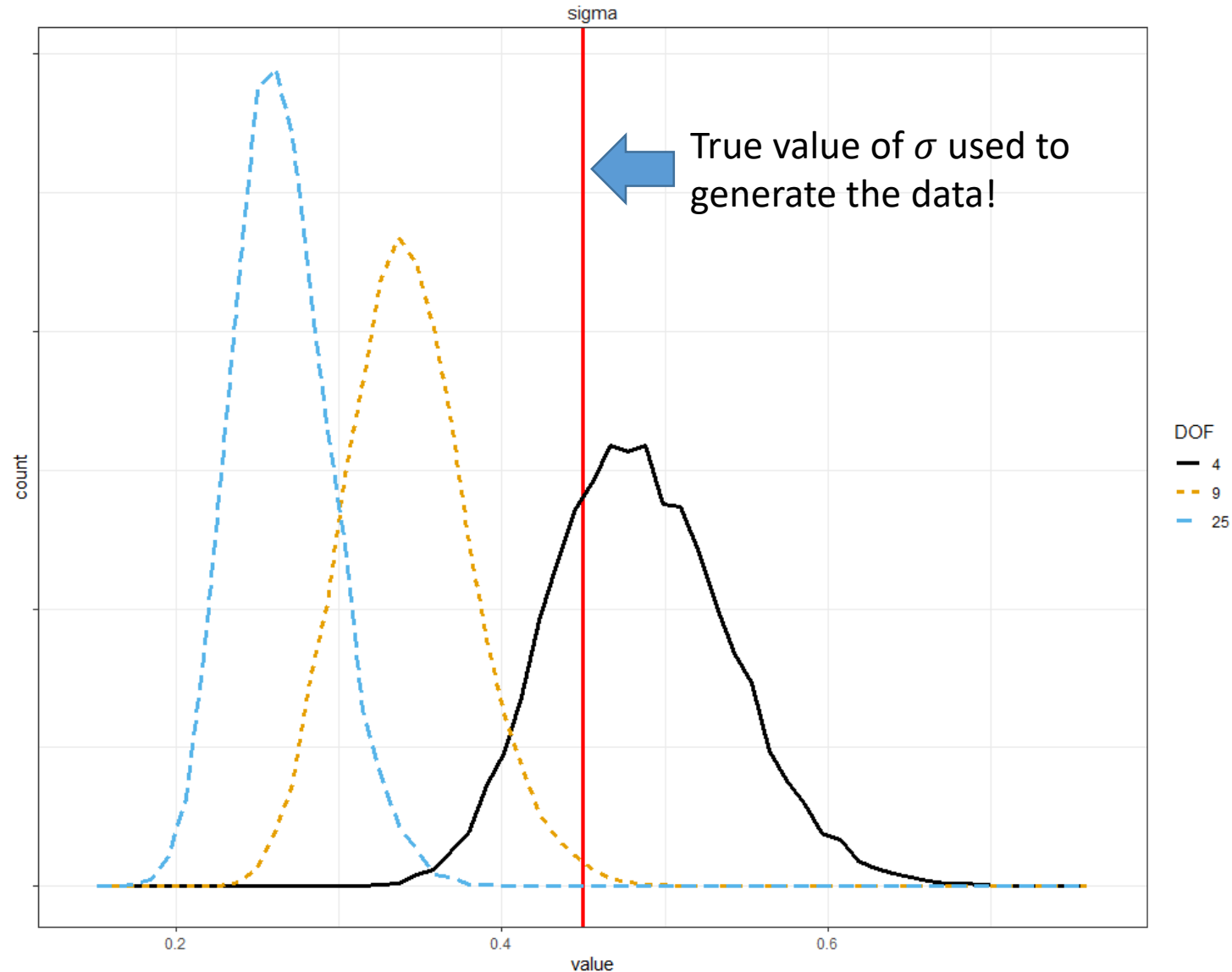
There are $26+1=27$ unknowns to learn!!!!!!

But running the Laplace Approximation is easy!!!!

Compare the posterior distributions on σ between the three models



What's happening to σ as the degree-of-freedom (COMPLEXITY) increases?



Let's make posterior predictions to help understand what's happening!

- We are interested in predicting at new input values, potentially on a test set.
- Assume that there are M prediction or test “locations”.
- Denote the test basis design matrix as Φ_* .
- How many rows and columns will be in Φ_* ?

Let's make posterior predictions to help understand what's happening!

• We are interested in predicting at new input values

Number of columns must be the same as in the training set! $\rightarrow J + 1$

Number of rows equals the number of prediction or test points: M

$$\Phi_* \text{ is } (M \times (J + 1))$$

- How many rows and columns will be in Φ_* ?

Predictions with β -parameter MLEs

- The MLE to the mean trend is the matrix multiplication of the test basis design matrix and coefficient MLE vector:

$$\boldsymbol{\mu}_{*,MLE} = \boldsymbol{\Phi}_* \boldsymbol{\beta}_{MLE} = \boldsymbol{\Phi}_* (\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \mathbf{y}$$

How do predictions work with our Bayesian model though?

- We generated $S = 10^4$ random posterior samples of the β -parameters and σ ...
- How can we make predictions?

Make $S = 10^4$ predictions!!!!!!

- Store the β posterior samples in a $(S \times (J + 1))$ matrix $\mathbf{B}$.
- Each row is a separate random set of β values!

$$\mathbf{B} = \begin{bmatrix} \beta_{s=1,j=0} & \beta_{s=1,j=1} & \cdots & \beta_{s=1,j=J} \\ \beta_{s=2,j=0} & \beta_{s=2,j=1} & \cdots & \beta_{s=2,j=J} \\ \vdots & \vdots & \ddots & \vdots \\ \beta_{s=S,j=0} & \beta_{s=S,j=1} & \cdots & \beta_{s=S,j=J} \end{bmatrix}$$

Make $S = 10^4$ predictions!!!!!!

- Store the β posterior samples in a $(S \times (J + 1))$ matrix $\mathbf{B}$.
- Take the transpose of B :

$$\mathbf{B}^T = \begin{bmatrix} \beta_{j=0,s=1} & \beta_{j=0,s=2} & \cdots & \beta_{j=0,s=S} \\ \beta_{j=1,s=1} & \beta_{j=1,s=2} & \cdots & \beta_{j=1,s=S} \\ \vdots & \vdots & \ddots & \vdots \\ \beta_{j=J,s=1} & \beta_{j=J,s=2} & \cdots & \beta_{j=J,s=S} \end{bmatrix}$$

What does each column of $\mathbf{B}^T$ correspond to?

$$\mathbf{B}^T = \begin{bmatrix} \beta_{j=0,s=1} & \beta_{j=0,s=2} & \cdots & \beta_{j=0,s=S} \\ \beta_{j=1,s=1} & \beta_{j=1,s=2} & \cdots & \beta_{j=1,s=S} \\ \vdots & \vdots & \ddots & \vdots \\ \beta_{j=J,s=1} & \beta_{j=J,s=2} & \cdots & \beta_{j=J,s=S} \end{bmatrix}$$

What does each column of $\mathbf{B}^T$ correspond to?

Each column is a random draw of all $J + 1$ coefficients!!

$$\mathbf{B}^T = \begin{bmatrix} \overbrace{\beta_{j=0,s=1}}^{\boldsymbol{\beta}_{s=1}} & \overbrace{\beta_{j=0,s=2}}^{\boldsymbol{\beta}_{s=2}} & \cdots & \overbrace{\beta_{j=0,s=S}}^{\boldsymbol{\beta}_{s=S}} \\ \beta_{j=1,s=1} & \beta_{j=1,s=2} & \cdots & \beta_{j=1,s=S} \\ \vdots & \vdots & \ddots & \vdots \\ \beta_{j=J,s=1} & \beta_{j=J,s=2} & \cdots & \beta_{j=J,s=S} \end{bmatrix}$$

We can therefore make $S = 10^4$ posterior predictions with matrix multiplication!

$$\mathbf{U}_* = \mathbf{\Phi}_* \mathbf{B}^T$$

We can therefore make $S = 10^4$ posterior predictions with matrix multiplication!

$$\mathbf{U}_* = \mathbf{\Phi}_* \mathbf{B}^T$$

$(M) \times (S)$

Each column of the $\mathbf{U}_*$ matrix is a posterior random draw of the mean trend!

Make a test grid of 101 evenly spaced points between the bounds of the training set

```
x_test <- seq(min(demo_sine$x)-0.05, max(demo_sine$x)+0.05, length.out = 101)
test_df <- data.frame(x = x_test)
```

We must create the test basis design matrix ourselves!

```
basis4 <- splines::ns(demo_sine$x, df = 4)

interior_knots_4 <- as.vector(attributes(basis4)$knots)

boundary_knots_4 <- as.vector(attributes(basis4)$Boundary.knots)

Xmat_test_4 <- model.matrix( ~ splines::ns(x,
                                         knots = interior_knots_4,
                                         Boundary.knots = boundary_knots_4),
                           data = test_df)

### force the same names as the training set design matrix
colnames(Xmat_test_4) <- colnames(Xmat4)
```

Predict the test set mean trend matrix

```
> dim(Xmat_test_4)
[1] 101    5
> Bmat4 <- post_samples_4 %>% select(starts_with("beta_")) %>% as.matrix()
> dim(Bmat4)
[1] 10000    5
> Umat4 <- Xmat_test_4 %*% t(Bmat4)
> dim(Umat4)
[1]    101 10000
```

Print out the dimensions of each of the matrices.
The test basis design matrix has 101 rows while the
posterior samples matrix has 10000 rows!

Predict the test set mean trend matrix

Isolate the posterior samples on the β -parameters,
convert to a matrix with `as.matrix()`

```
> [1] 101      5  
> Bmat4 <- post_samples_4 %>% select(starts_with("beta_")) %>% as.matrix()  
> dim(Bmat4)  
[1] 10000      5  
> Umat4 <- Xmat_test_4 %*% t(Bmat4)  
> dim(Umat4)  
[1] 101 10000
```

Predict the test set mean trend matrix

Matrix multiplication of the mean trend!

```
> dim(Bmat4)
```

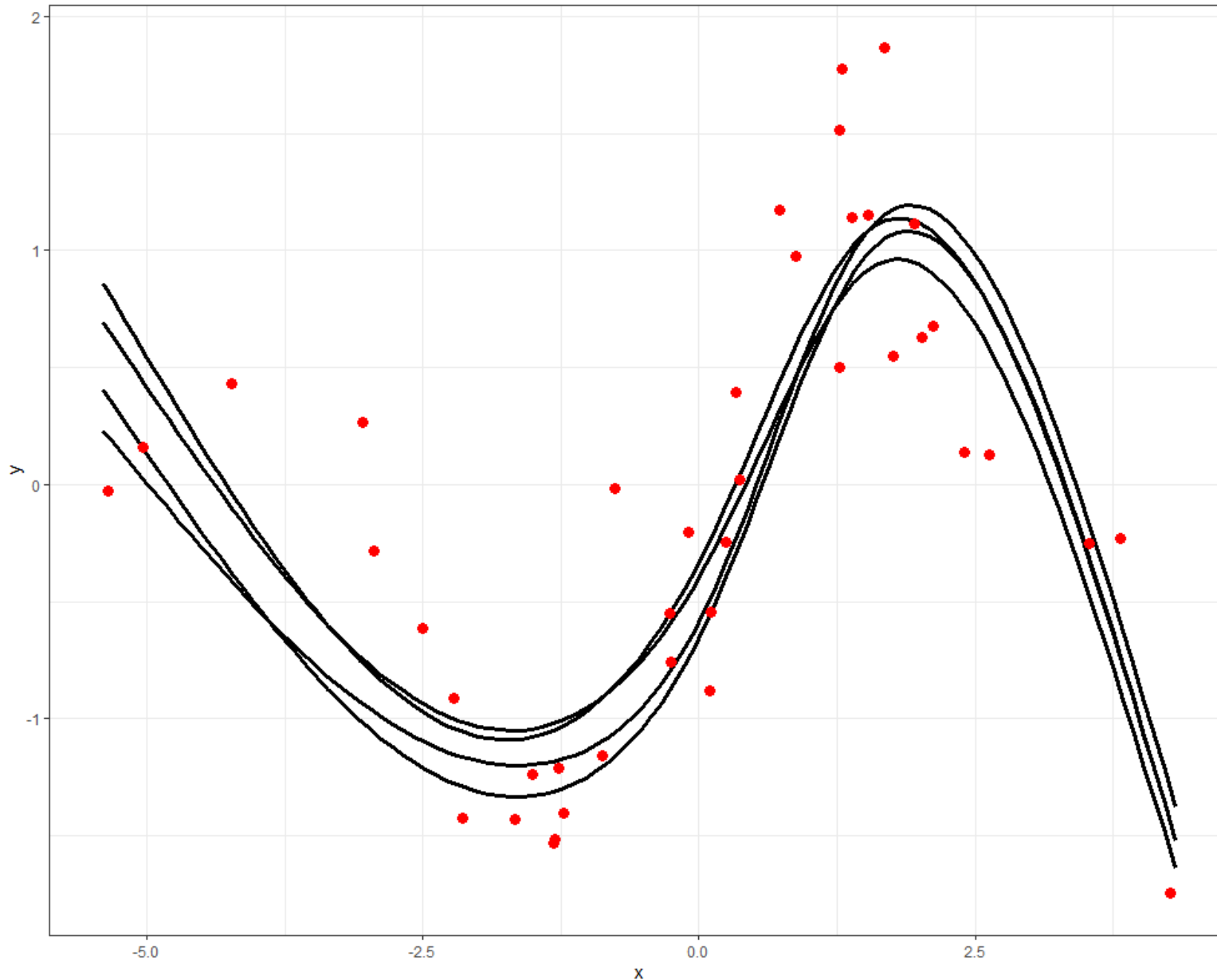
```
[1] 10000 5
```

```
> Umat4 <- Xmat_test_4 %*% t(Bmat4)
```

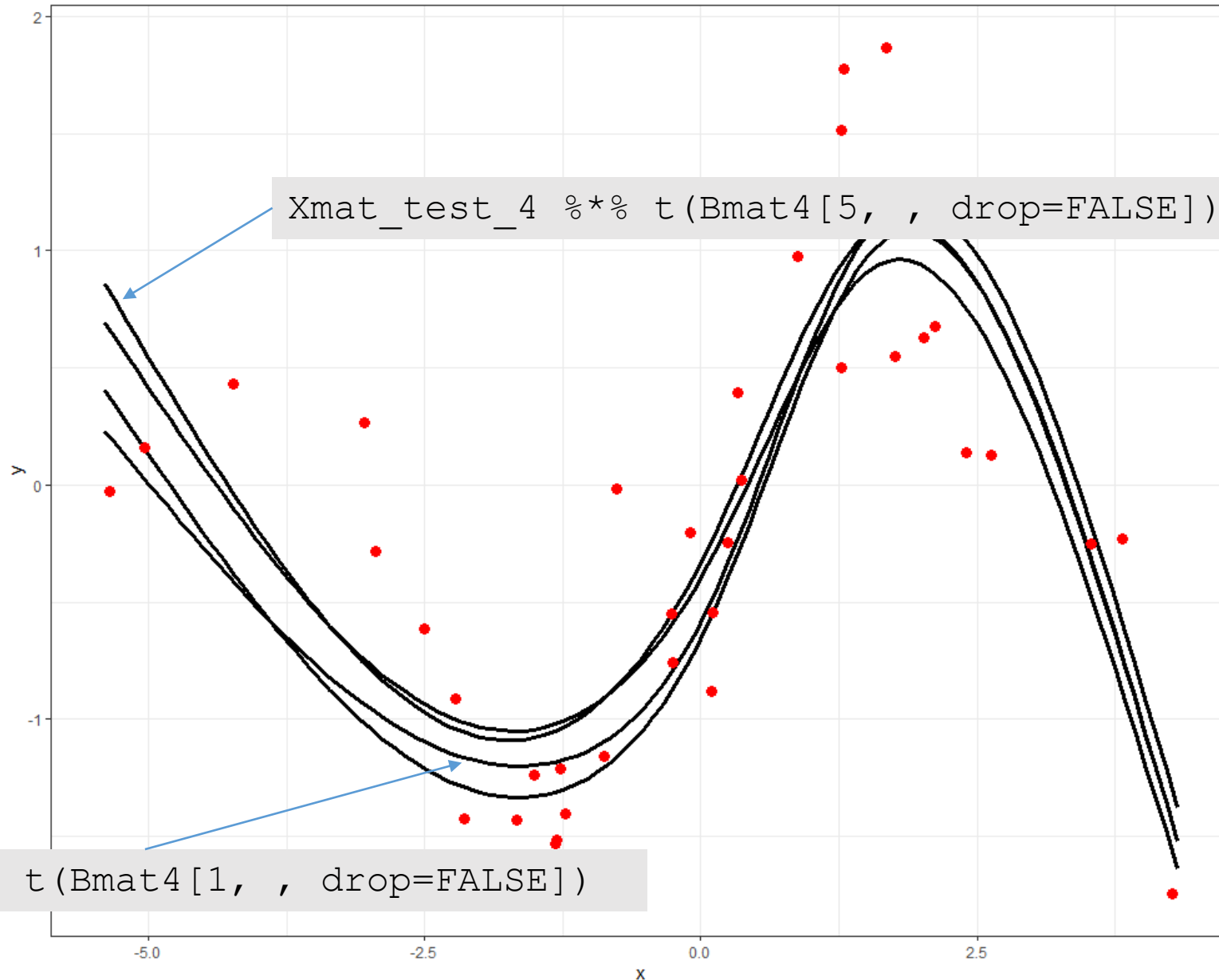
```
> dim(Umat4)
```

```
[1] 101 10000
```

4 random mean trends with respect to x

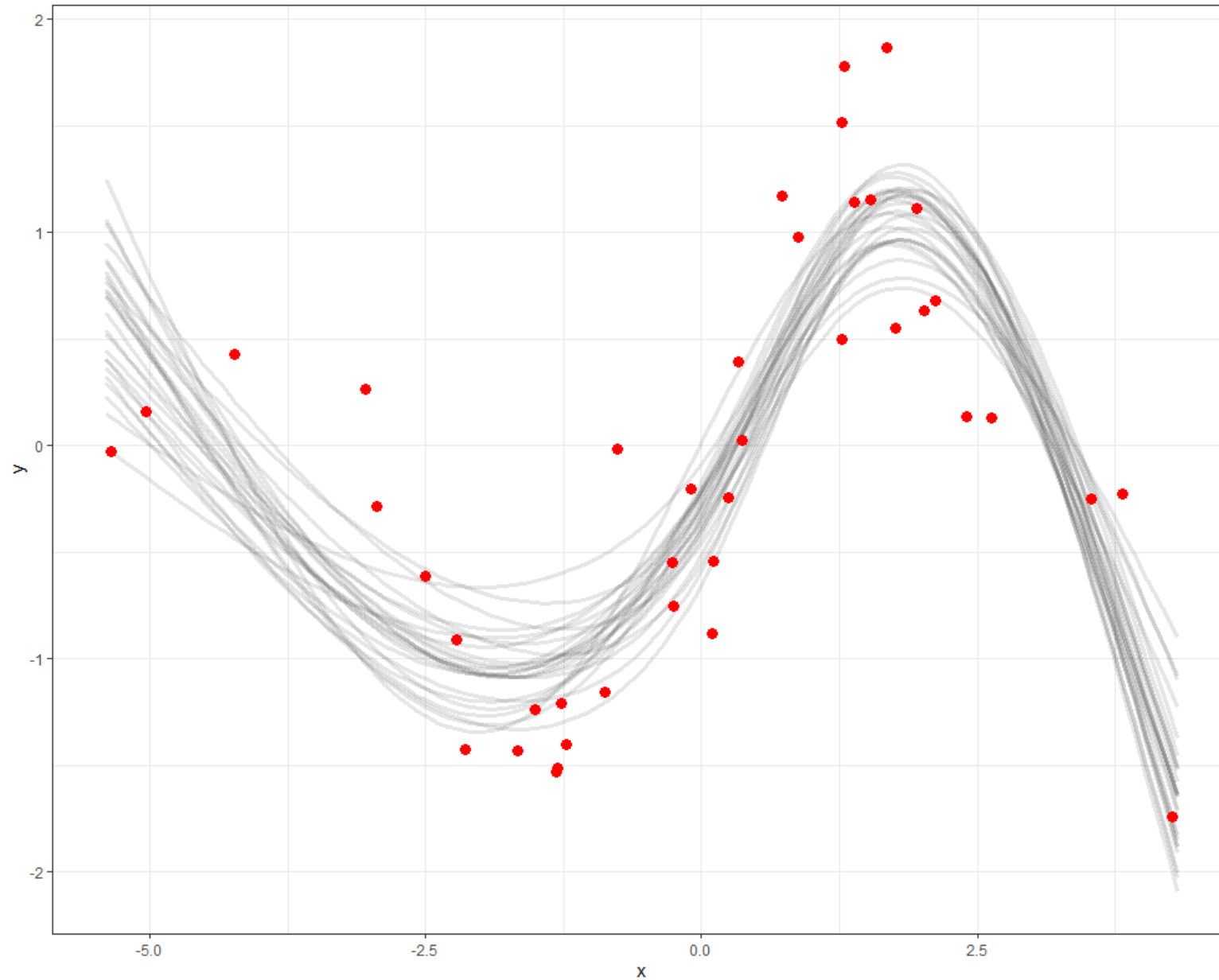


4 random mean trends with respect to x

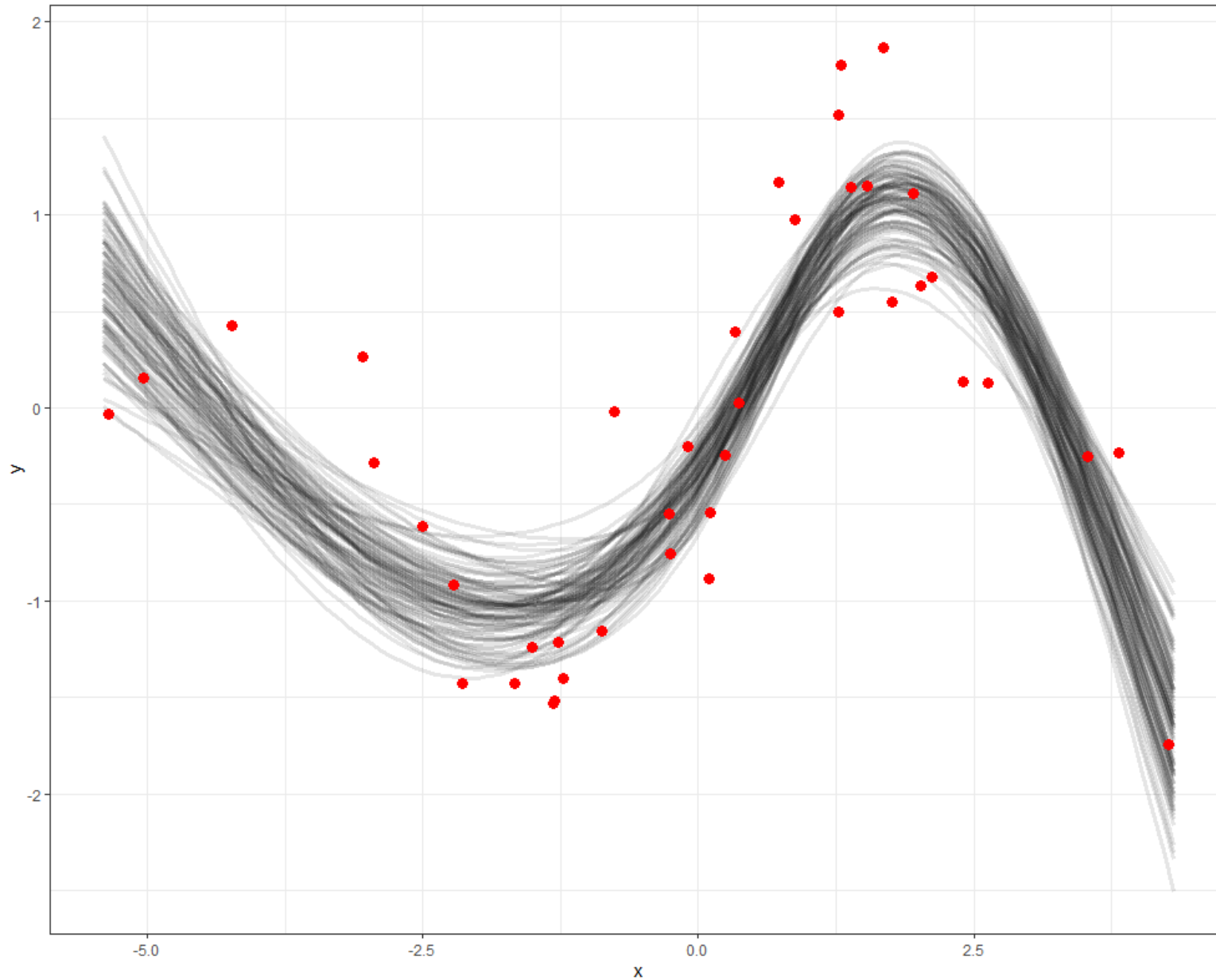


Each curve is the mean trend associated with one set of coefficients β_s

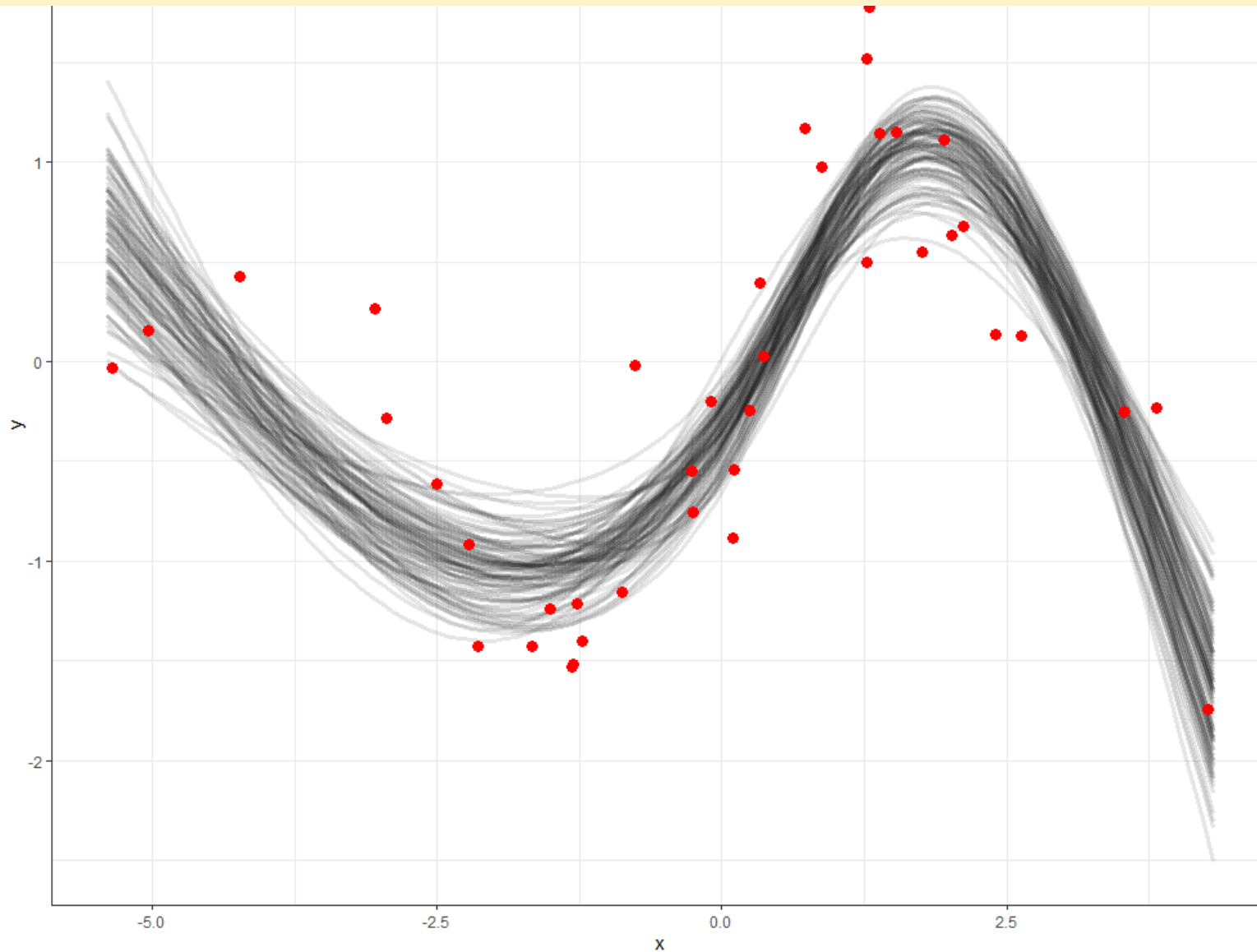
25 random mean trends with respect to x



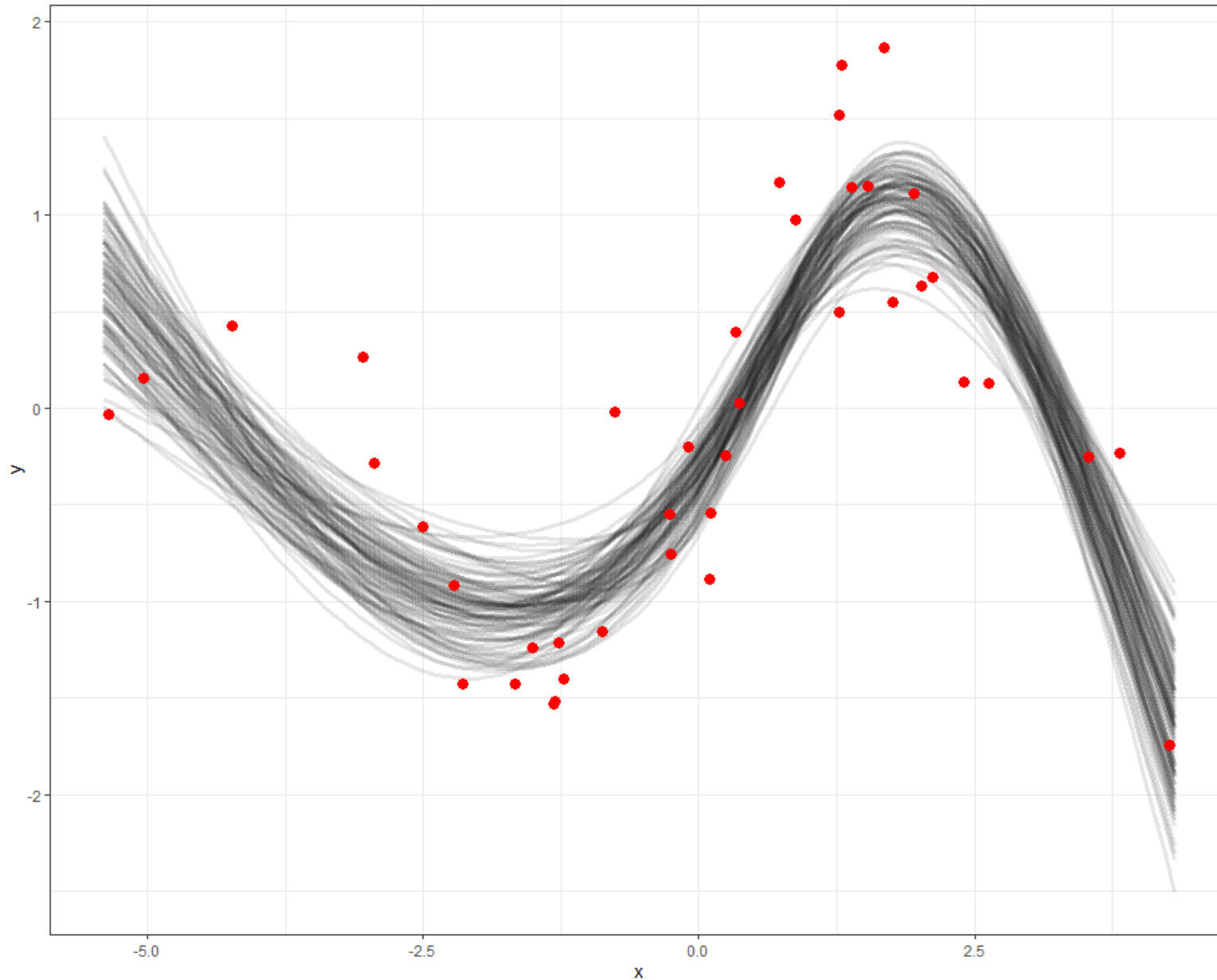
100 random mean trends with respect to x



What are we visualizing? As we continue to add more and more curves...what are we representing?

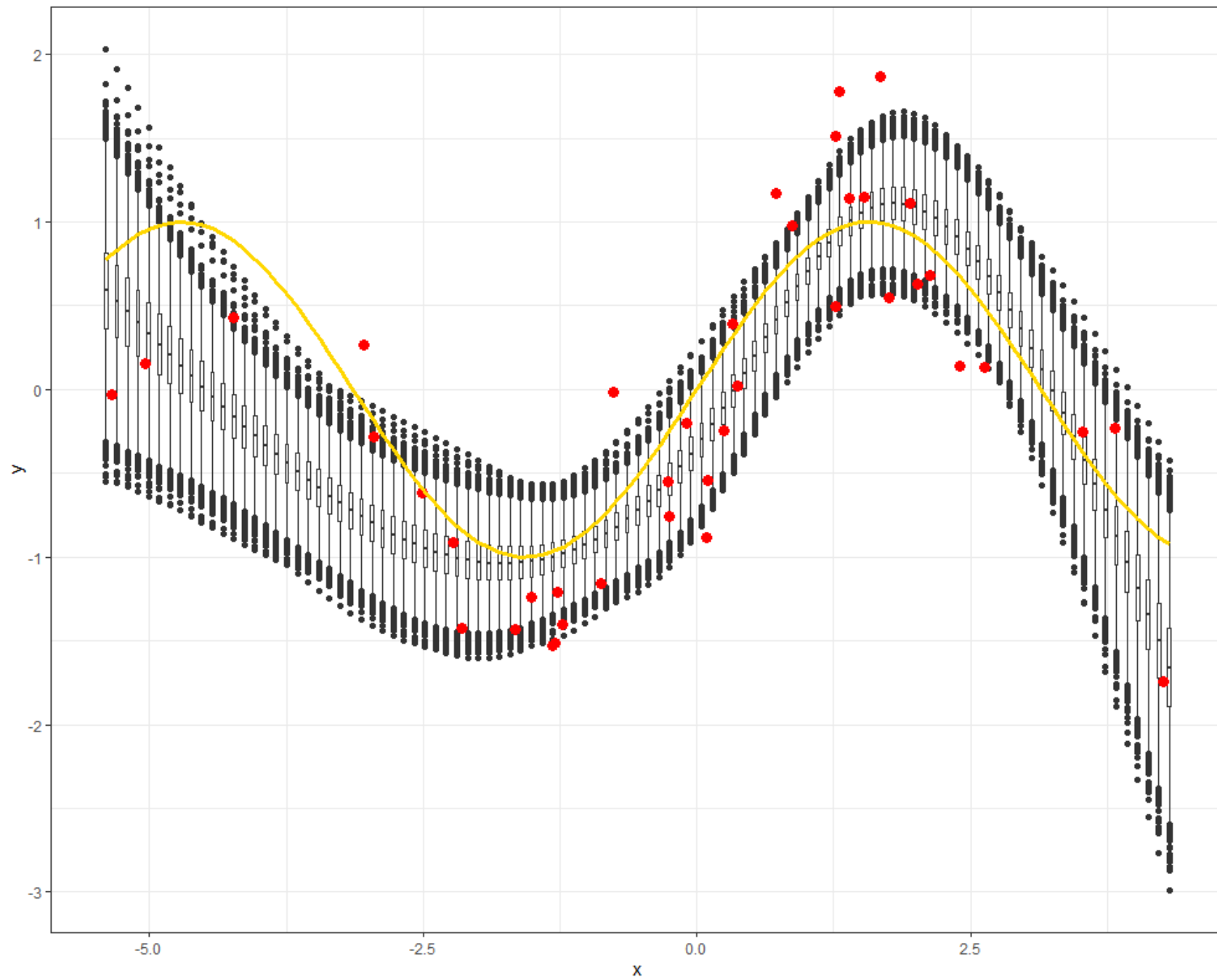


The posterior predictive distribution of the mean trend!



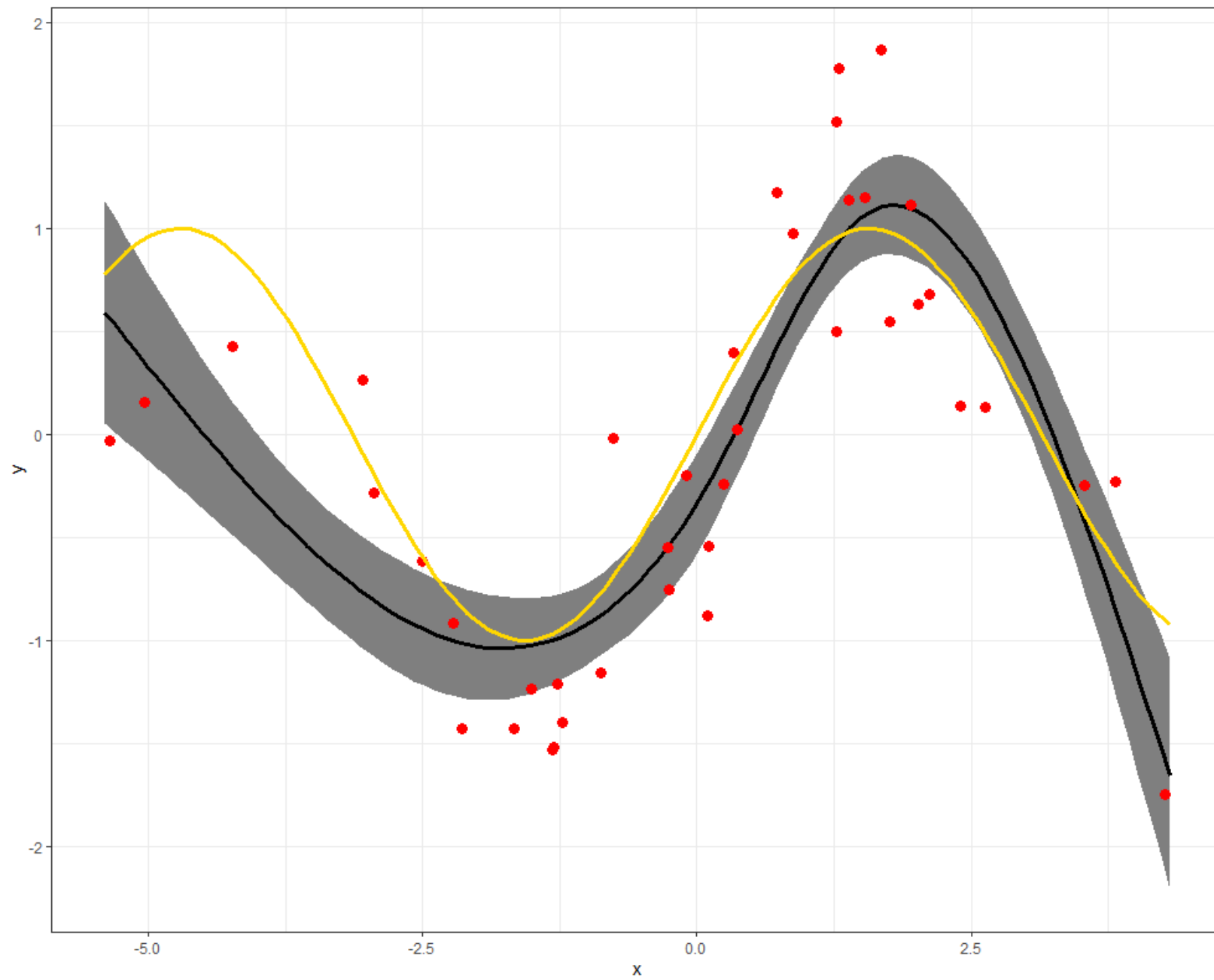
We can summarize all 10^4 mean trends with respect to the input x

- One way to summarize is to use boxplots.
- Display the median, 25th and 75th quantiles by a box.
- All “non-outlier” points with whiskers.



I prefer to summarize the posterior predictive distributions with ribbons

- In the following figure, the black line corresponds to the posterior mean of the mean trend.
- The grey ribbon represents the middle 90% uncertainty interval on the mean.



Where does the uncertainty in the mean come from?

- Remember we calculated 10^4 curves from the matrix operation:

$$\mathbf{U}_* = \mathbf{\Phi}_* \mathbf{B}^T$$

- We have 10^4 curves because we have 10^4 possible values for the β -parameters.
- The uncertainty in the β -parameters induces uncertainty in the mean trend!!

Uncertainty in the mean trend is usually referred to as
the CONFIDENCE INTERVAL

But...that is not the only form of uncertainty in our model...

- The mean trend is the DETERMISTIC portion of the model...
- **We also have the STOCHASTIC portion in the LIKELIHOOD itself!**

$$y_n \mid \mu_n, \sigma \sim \text{normal}(y_n \mid \mu_n, \sigma)$$

- Even if we knew σ perfectly, **we are uncertain about the value of the response**. It's normally distributed around the mean!

Uncertainty in σ induces “extra uncertainty” in our uncertainty of the response.

Let's generate random samples of the response

- We need to make use of the z-score “trick” to efficiently generate the random observations.

$$x = \mu + \sigma \times z$$

$$z \sim \text{normal}(z \mid 0, 1)$$

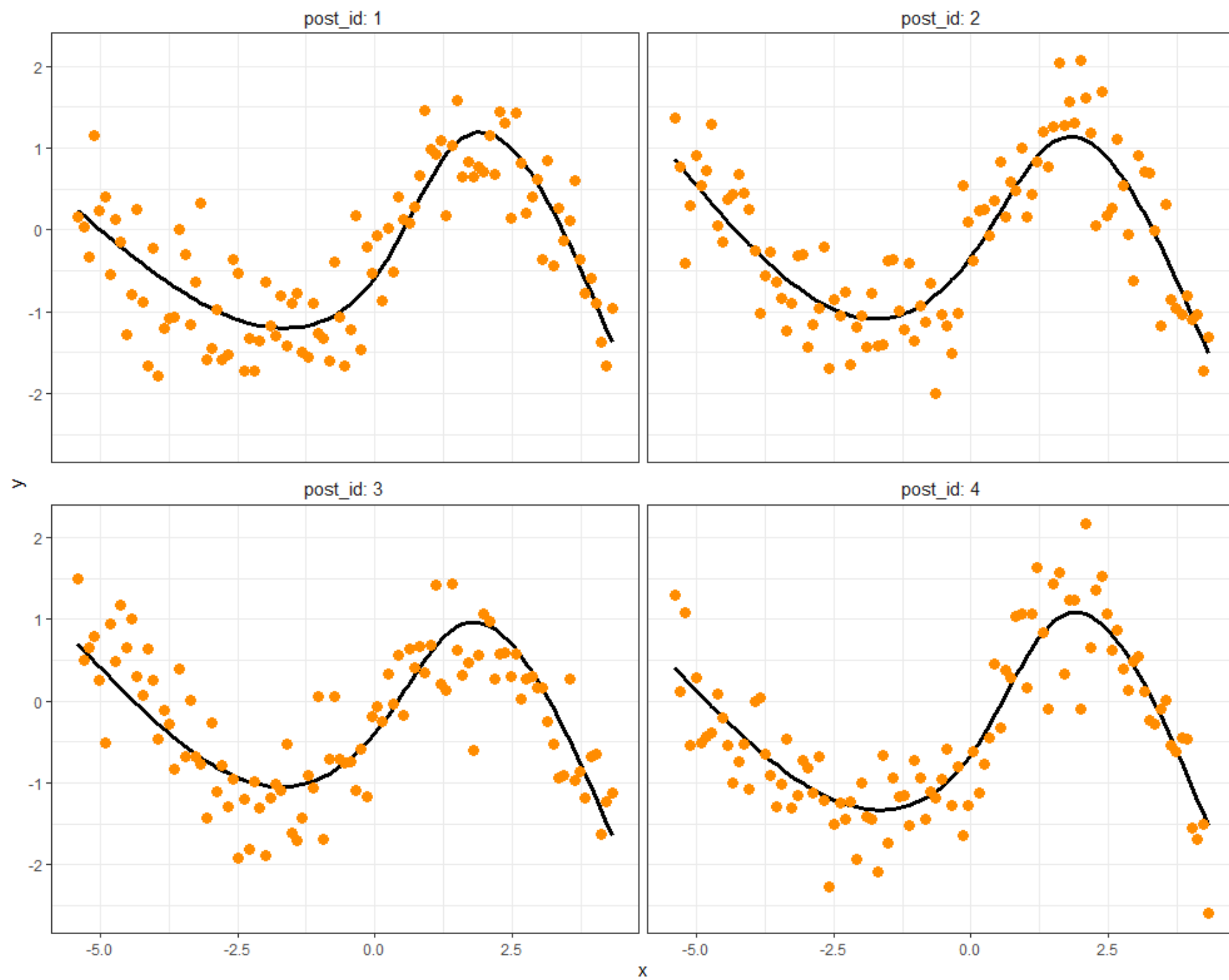
- Can generate random values from an arbitrary Gaussian by generating random values from a standard normal and rescaling.

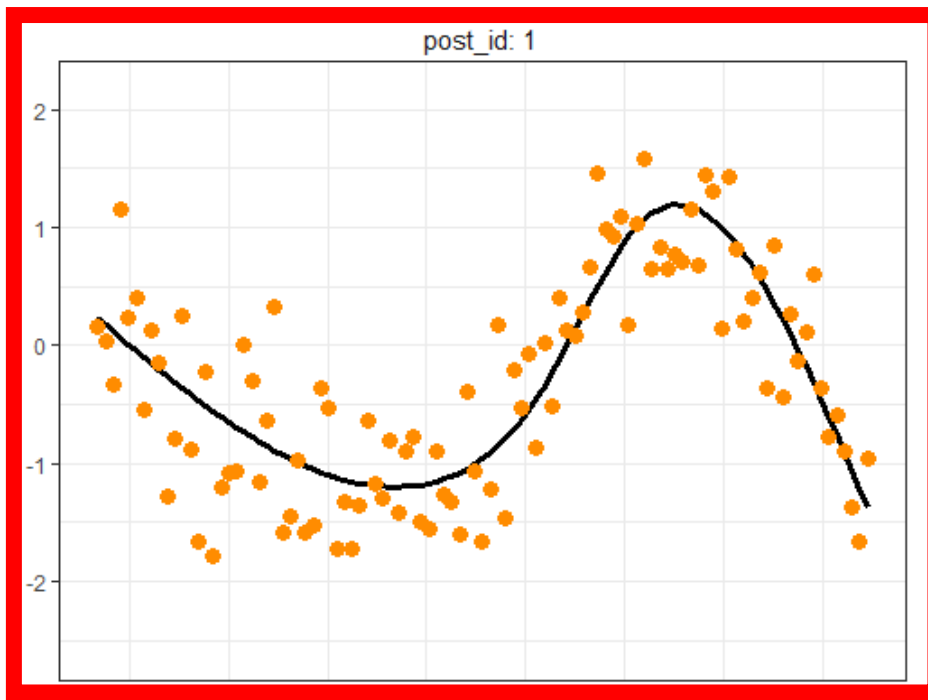
In code, those steps are...

```
> ### add in the effect of the noise
> noise_vec_4 <- post_samples_4 %>% pull(sigma)
> ### reshape the posterior samples into a matrix
> Rmat4 <- matrix(rep(noise_vec_4, nrow(Xmat_test_4)), nrow(Xmat_test_4), byrow = TRUE)
> dim(Rmat4)
[1] 101 10000
> # generate standard normal and assemble into matrix
> set.seed(13123)
> Zmat4 <- matrix(rnorm(nrow(Xmat_test_4)*nrow(Bmat4)), nrow(Xmat_test_4), byrow = TRUE)
> # calculate the random observation predictions
> Ymat4 <- Umat4 + Rmat4 * Zmat4
> dim(Ymat4)
[1] 101 10000
```


Remember, the likelihood is sampled for each posterior sample of β and σ

- In the following figure, each subplot is a **posterior sample**.
- The **posterior samples** are denoted as `post_id`.
- The black curves are random mean trend and the orange dots are the randomly generated “observations” around the mean.



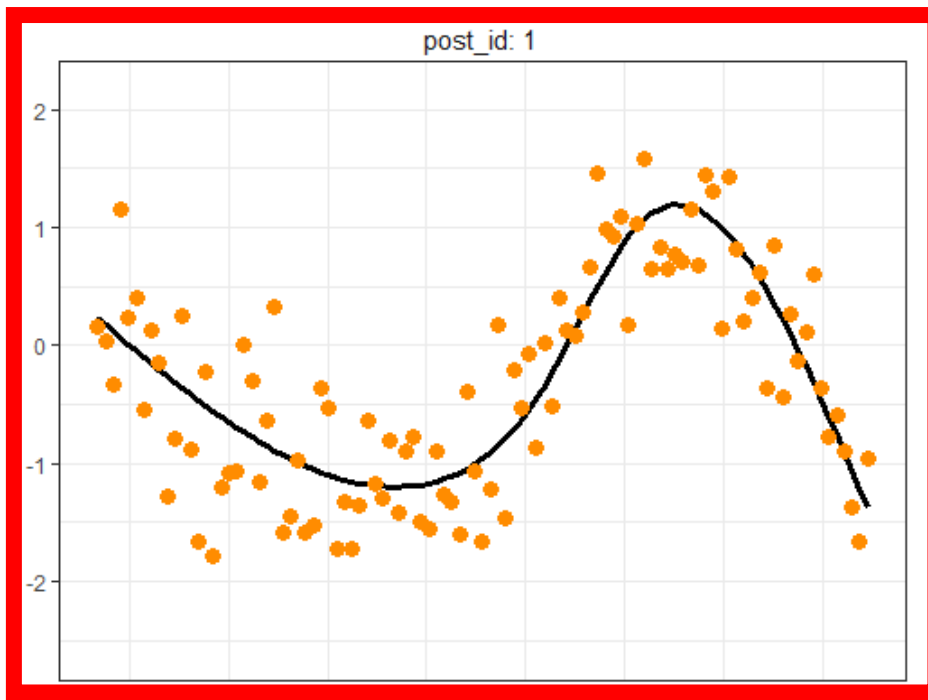


The orange dots can be generated for this specific posterior sample using `rnorm()`:

```
rnorm(n=nrow(Xmat_test_4),
      mean = as.vector(Xmat_test_4 %*% t(Bmat4[1, , drop=FALSE])),
      sd = noise_vec_4[1])
```

Or in math notation for the m -th prediction point:

$$\text{normal}(y_{*m} \mid \phi_{*m,:} \boldsymbol{\beta}_{s=1}, \sigma_{s=1})$$



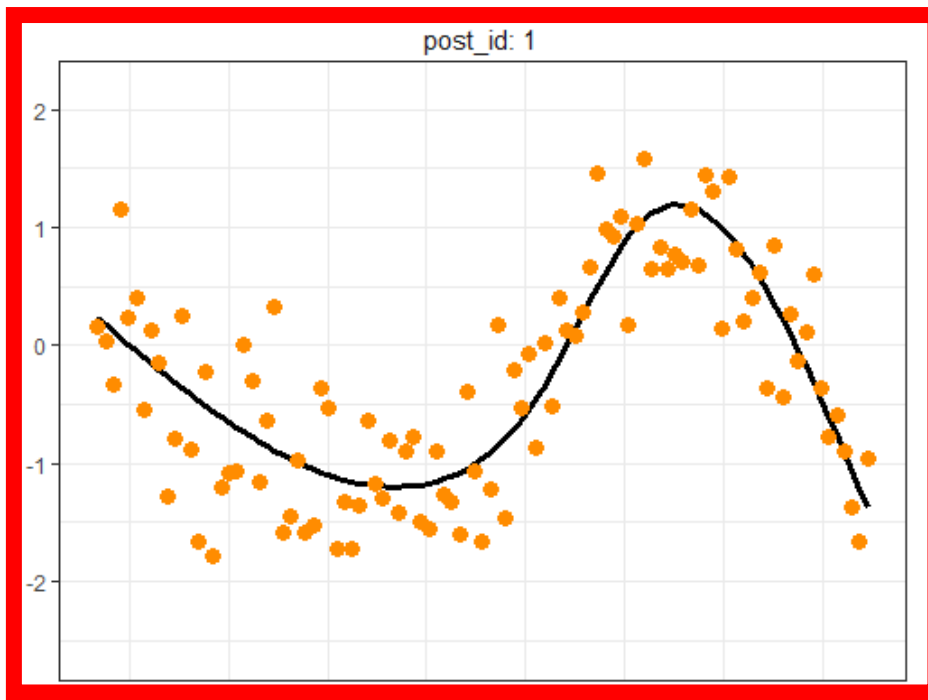
The orange dots can be generated for this specific posterior sample using `rnorm()`:

```
rnorm(n=nrow(Xmat test 4),  
      mean = as.vector(Xmat test 4 %*% t(Bmat4[1, , drop=FALSE])),  
      sd = noise_vec_4[1])
```

Mean trend for **ONE** set of β -parameters!

Or in math notation for the m -th prediction point:

$$\text{normal}(y_{*m} \mid \phi_{*m,:} \beta_{s=1}, \sigma_{s=1})$$



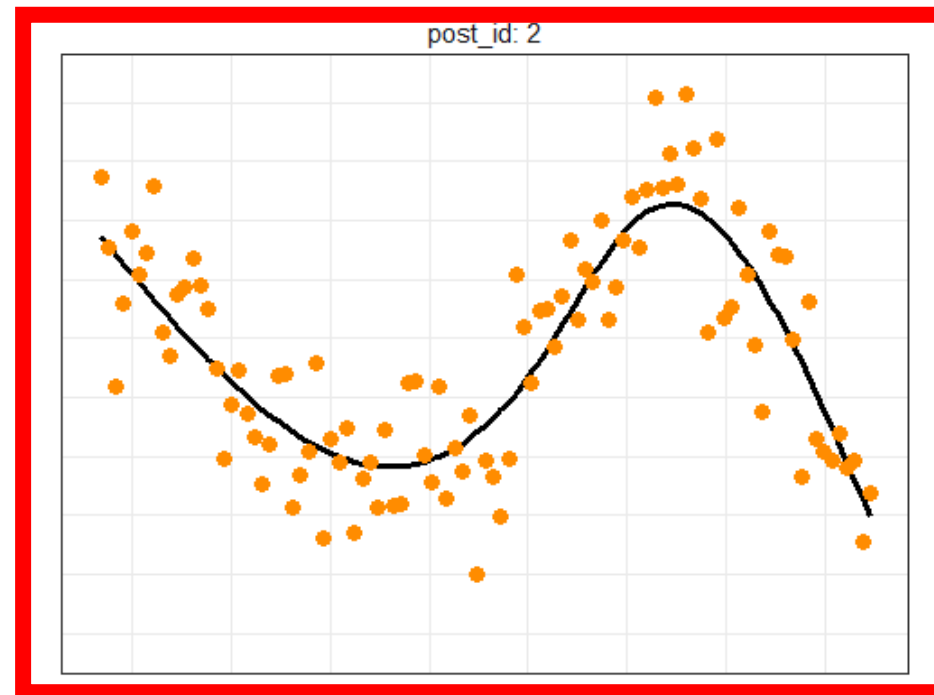
The orange dots can be generated for this specific posterior sample using `rnorm()`:

```
rnorm(n=nrow(Xmat_test_4),
      mean = as.vector(Xmat_test_4 %*% t(Bmat4[1, , drop=FALSE])),
      sd = noise_vec_4[1])
```

The σ parameter does **NOT** depend on the input!

Or in math notation for the m -th prediction point:

$$\text{normal}(y_{*m} \mid \phi_{*m,:} \boldsymbol{\beta}_{s=1}, \sigma_{s=1})$$

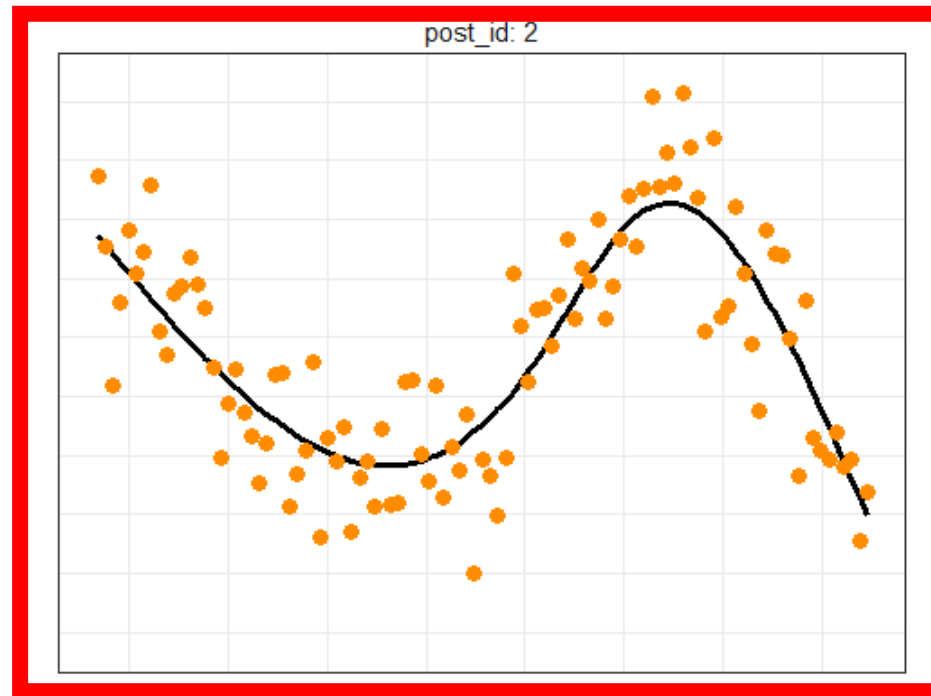


The orange dots can be generated for this specific posterior sample using `rnorm()`:

```
rnorm(n=nrow(Xmat_test_4),
      mean = as.vector(Xmat_test_4 %*% t(Bmat4[2, , drop=FALSE])),
      sd = noise_vec_4[2])
```

Or in math notation for the m -th prediction point:

$$\text{normal}(y_{*m} \mid \boldsymbol{\phi}_{*m, :} \boldsymbol{\beta}_{s=2}, \sigma_{s=2})$$



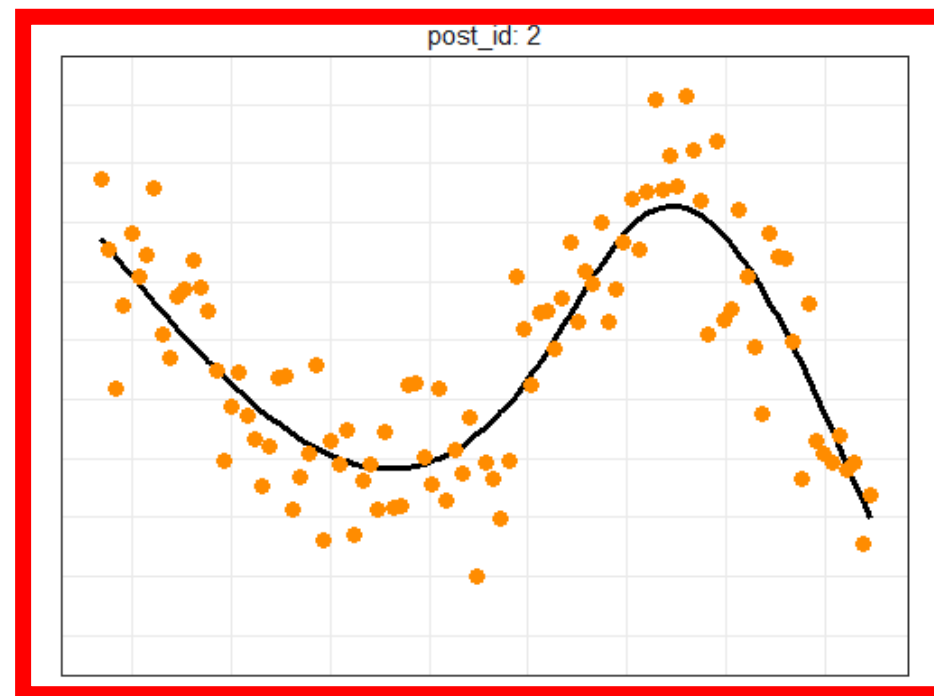
The orange dots can be generated for this specific posterior sample using `rnorm()`:

```
rnorm(n=nrow(Xmat test 4),  
      mean = as.vector(Xmat test 4 %*% t(Bmat4[2, , drop=FALSE])),  
      sd = noise_vec_4[2])
```

Mean trend for **ONE** set of β -parameters!

Or in math notation for the m -th prediction point:

$$\text{normal}(y_{*m} \mid \phi_{*m,:} \beta_{s=2}, \sigma_{s=2})$$



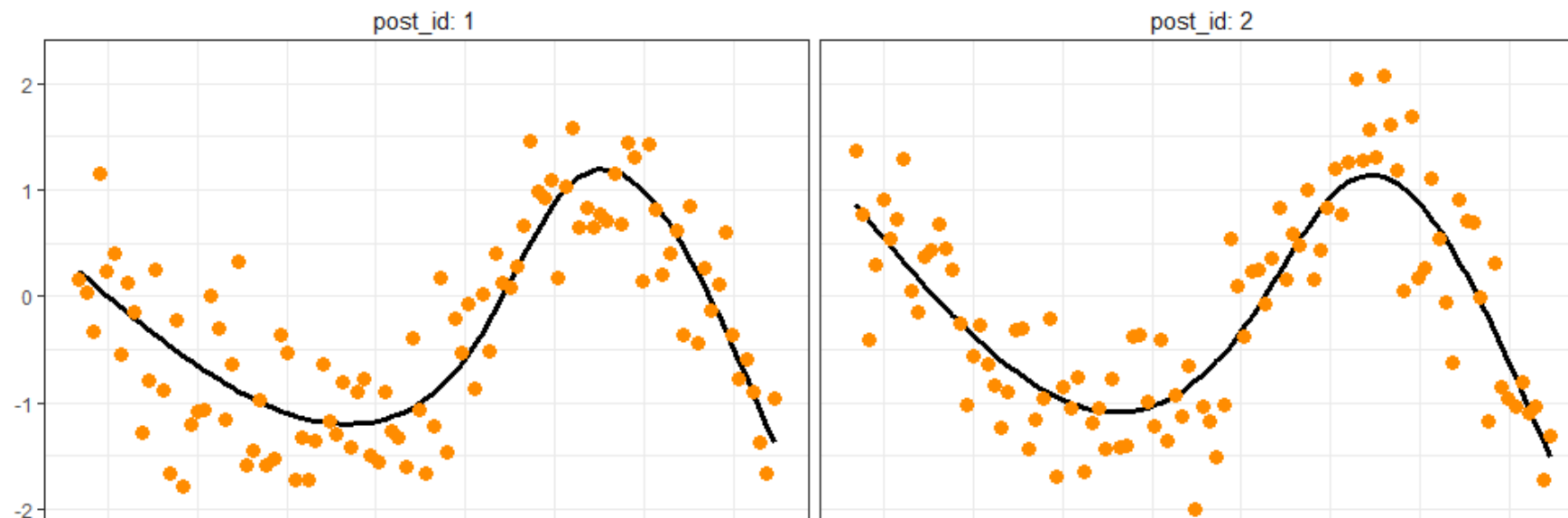
The orange dots can be generated for this specific posterior sample using `rnorm()`:

```
rnorm(n=nrow(Xmat_test_4),
      mean = as.vector(Xmat_test_4 %*% t(Bmat4[2, , drop=FALSE])),
      sd = noise_vec_4[2])
```

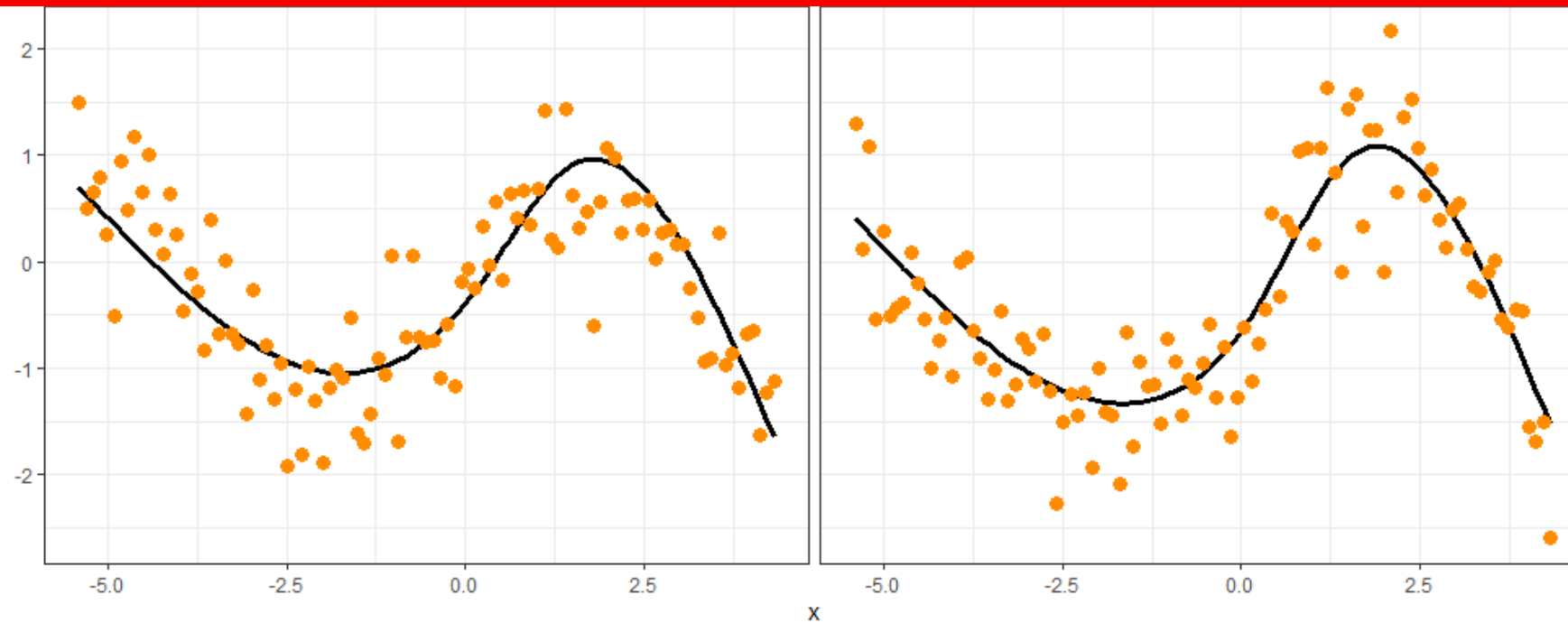
The σ parameter does **NOT** depend on the input!

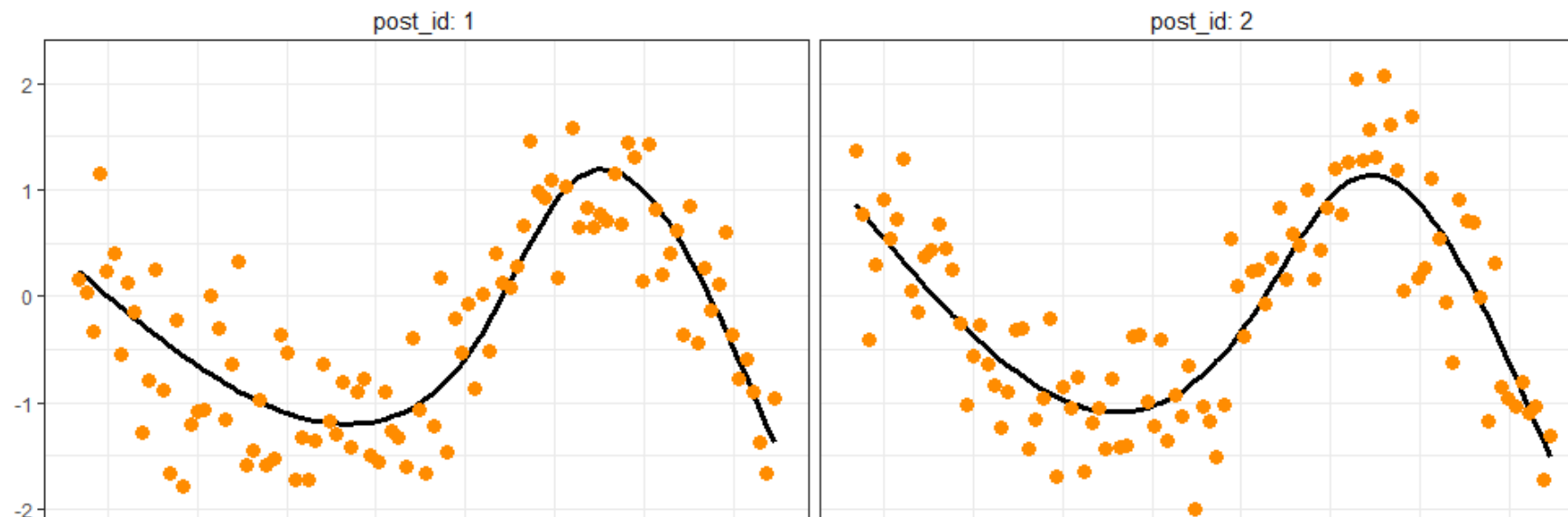
Or in math notation for the m -th prediction point:

$$\text{normal}(y_{*m} \mid \phi_{*m,:} \boldsymbol{\beta}_{s=2}, \sigma_{s=2})$$

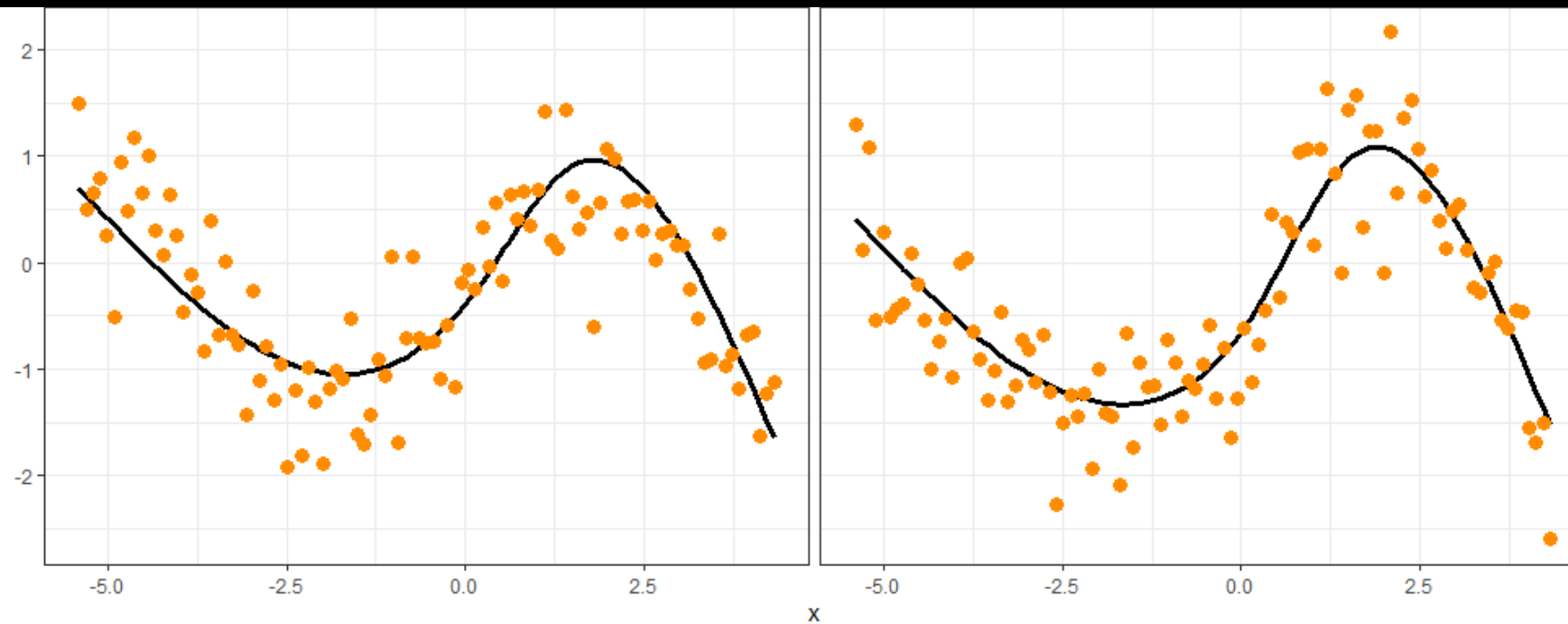


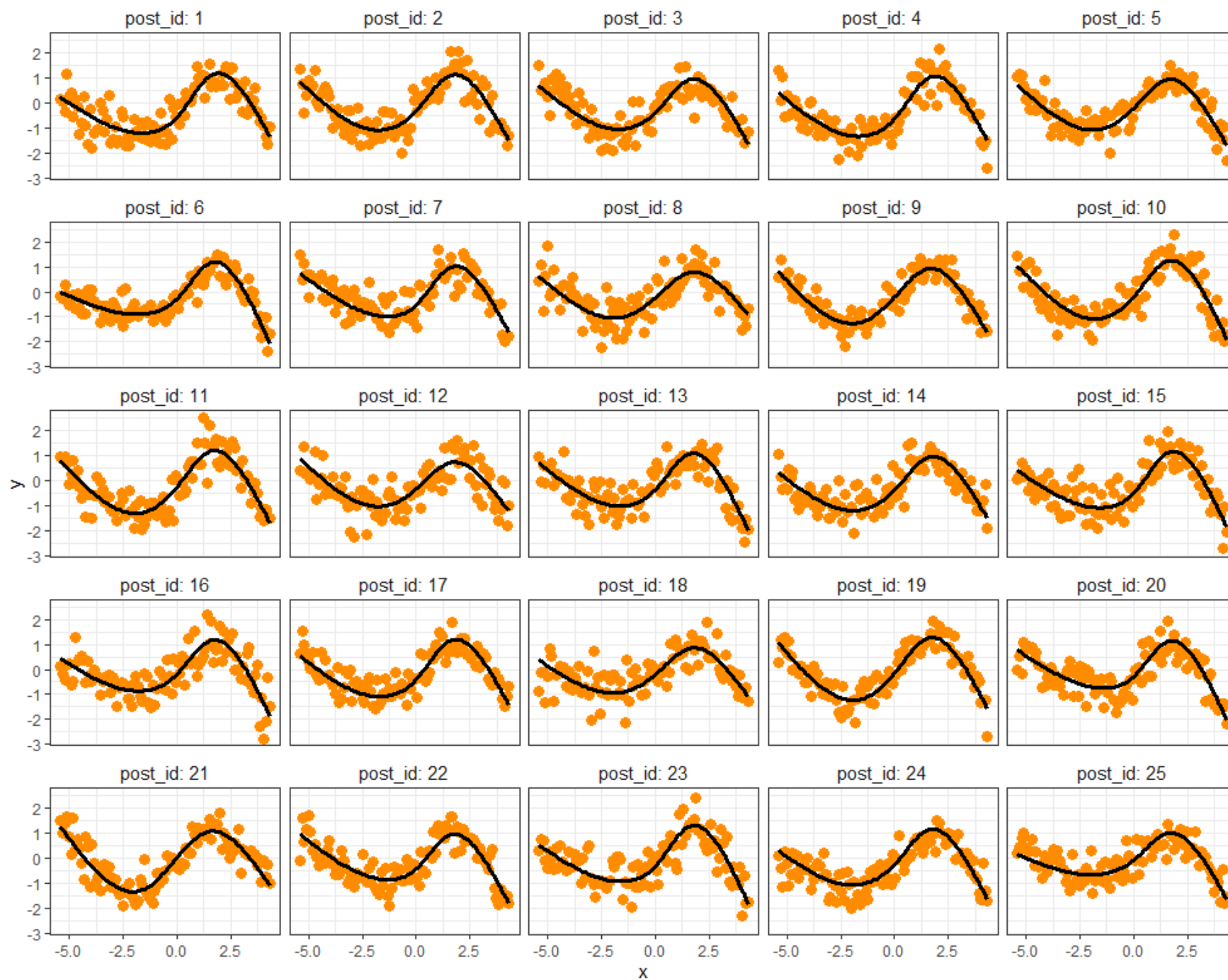
The matrix operations shown several slides ago generate all random samples for all posterior samples at all prediction points at once!





Why are we looking at these figures?
What do these plots tell us?





We can keep going...but ultimately we will summarize the **posterior predictive distribution**

- Two sets of summaries are calculated, the first we have already discussed, the **summaries on the mean trend**.
- The second summarizes the **response predictive distribution**.
- It accounts for the **error** or noise in the model and **represents the uncertainty in a response**, not the mean trend!

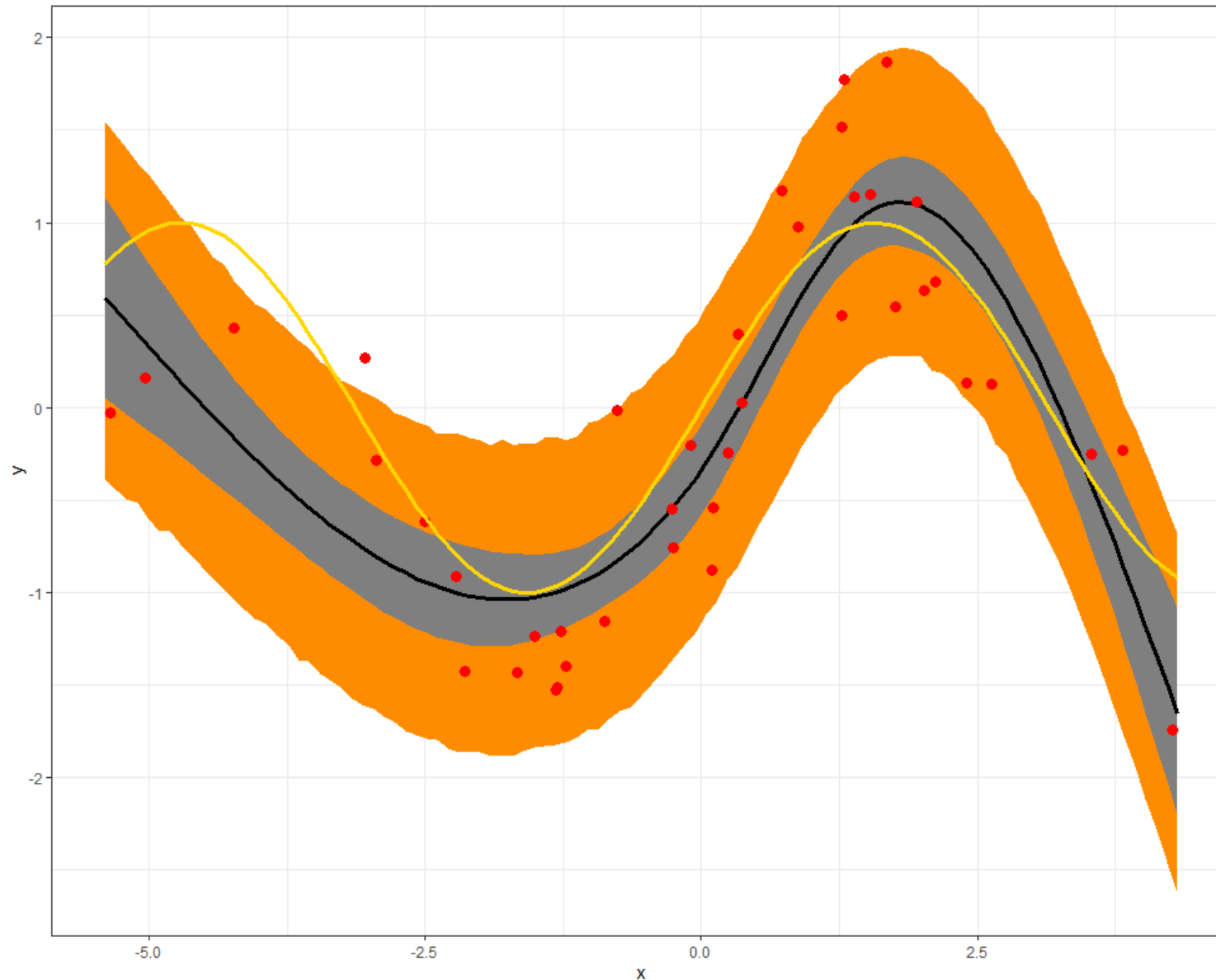
- The response predictive uncertainty intervals are referred to as **PREDICTION INTERVALS**.

Summarize by calculating the average, 5th and 95th quantiles.

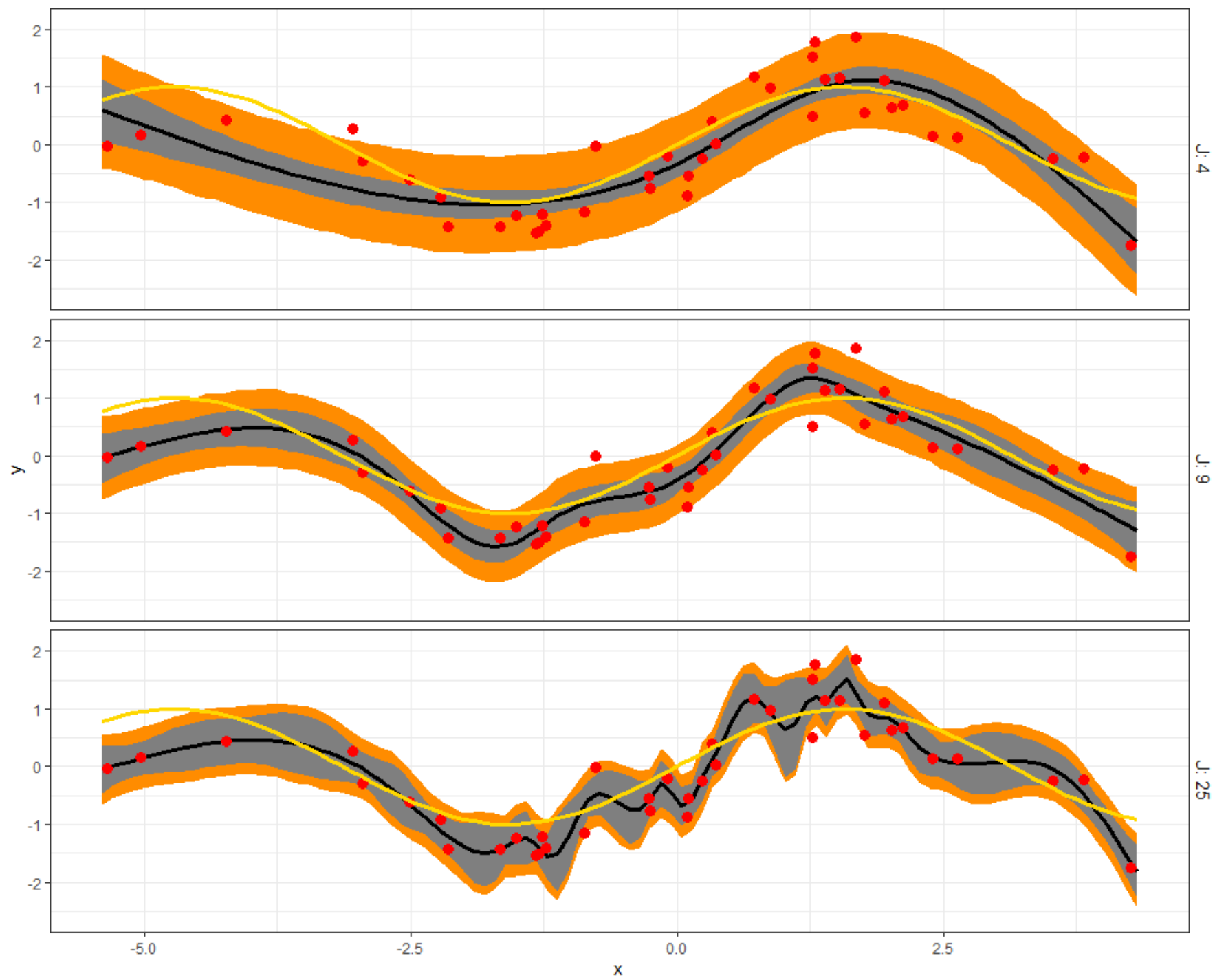
```
### summarize the posterior samples on the mean and response
mu_avg_4 <- rowMeans(Umat4)
mu_q05_4 <- apply(Umat4, 1, stats::quantile, probs = 0.05)
mu_q95_4 <- apply(Umat4, 1, stats::quantile, probs = 0.95)

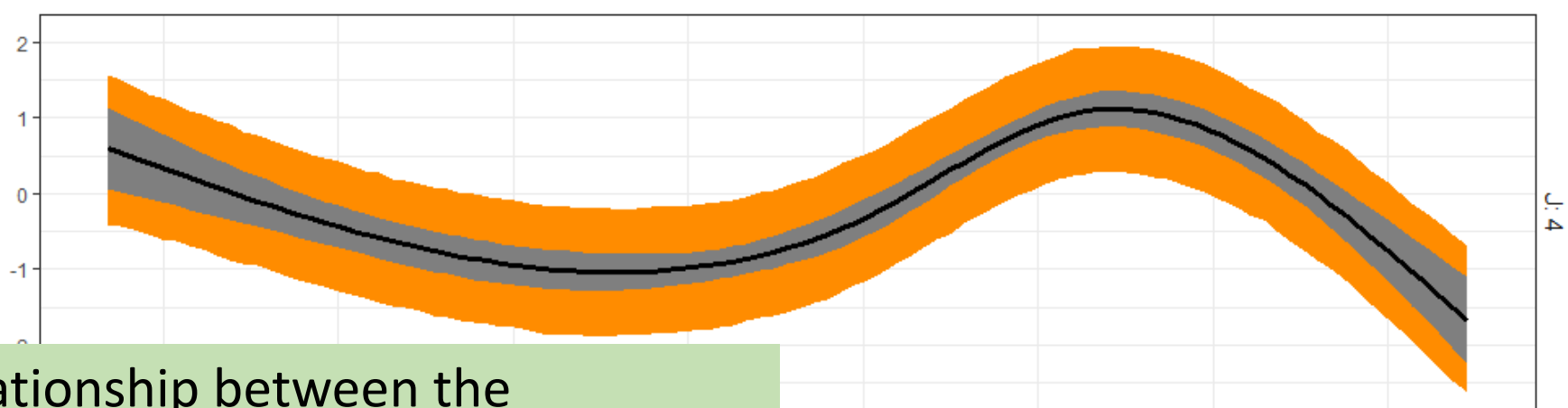
# summarize the posterior observation predictions
y_avg_4 <- rowMeans(Ymat4)
y_q05_4 <- apply(Ymat4, 1, stats::quantile, probs = 0.05)
y_q95_4 <- apply(Ymat4, 1, stats::quantile, probs = 0.95)
```

Mean trend CONFIDENCE INTERVAL and response PREDICTION INTERVAL compared to the true trend and observations

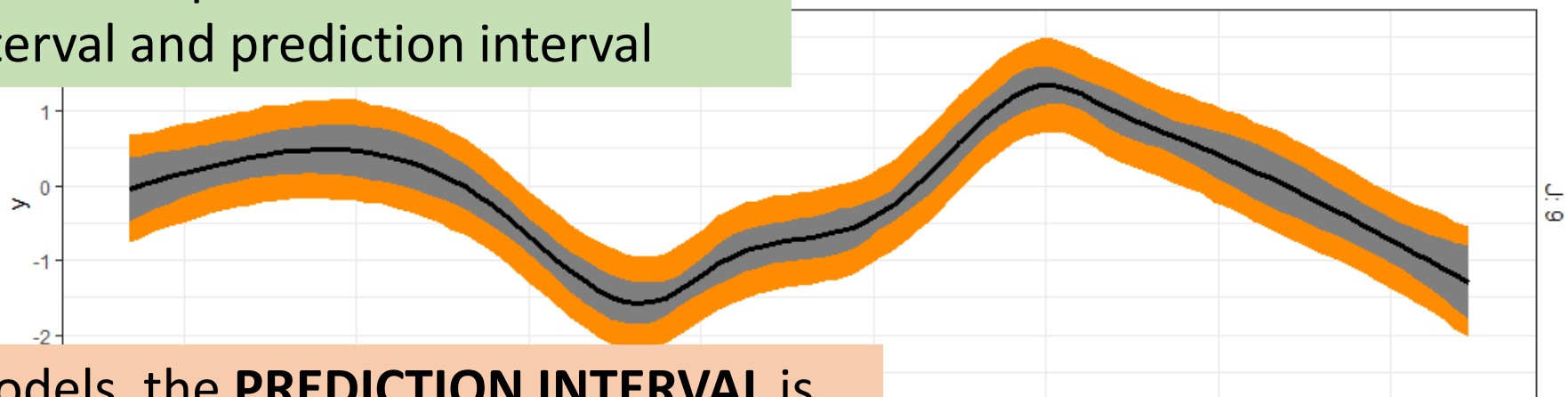


How do the predictions change as we increase the spline degree-of-freedom?

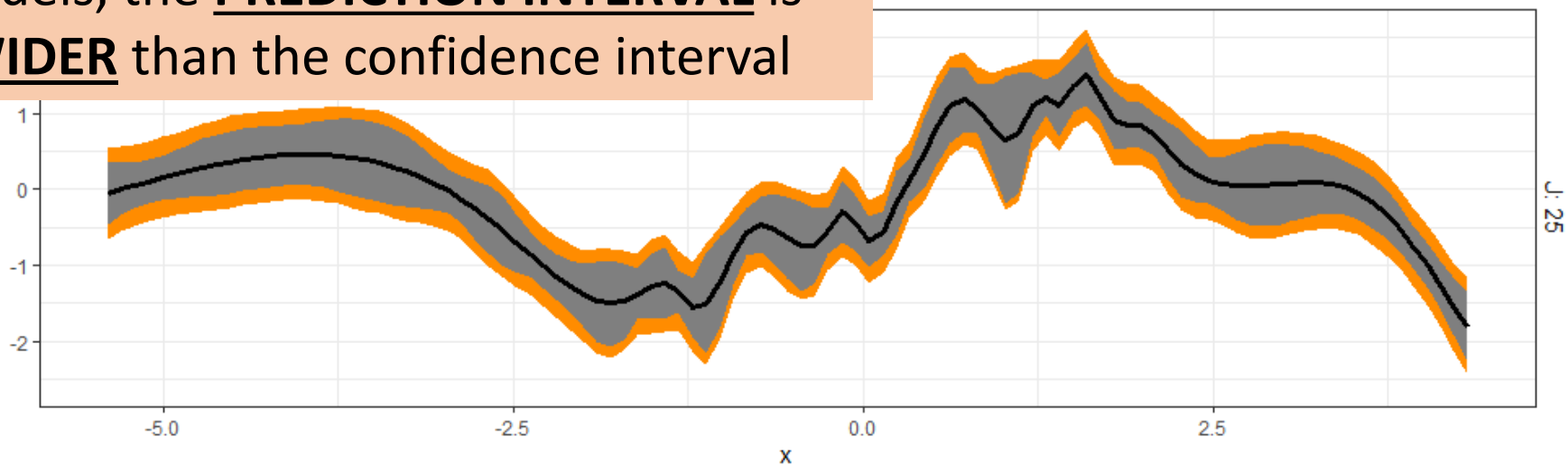


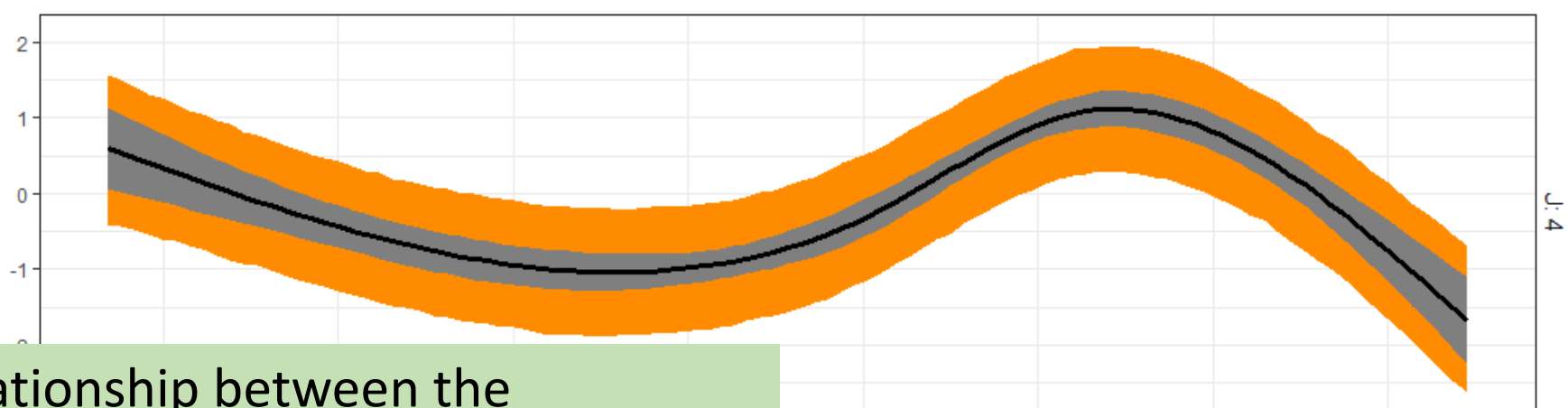


Look at the relationship between the confidence interval and prediction interval

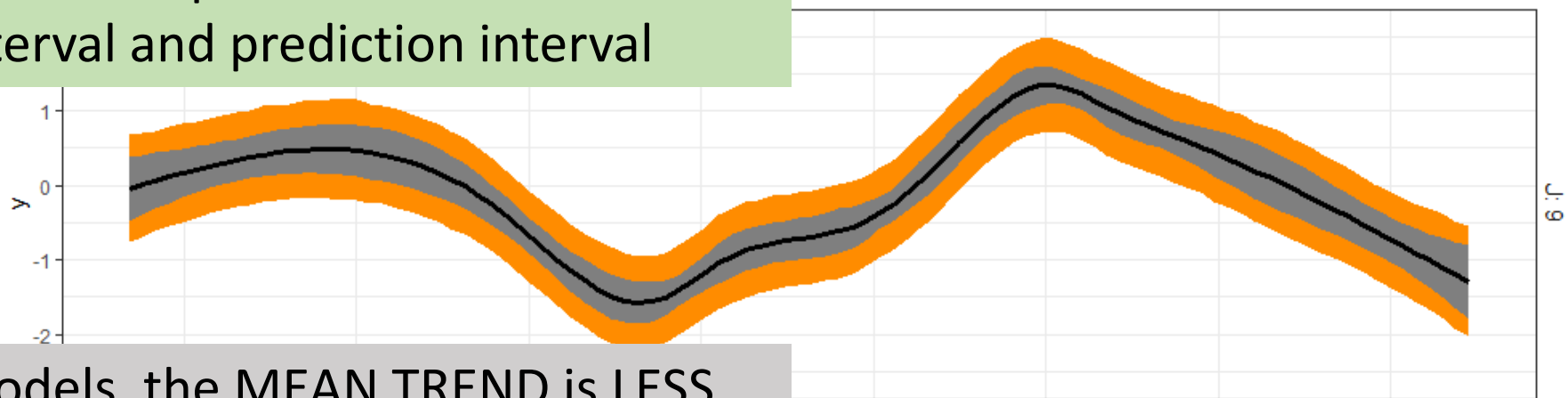


For simpler models, the PREDICTION INTERVAL is substantially WIDER than the confidence interval

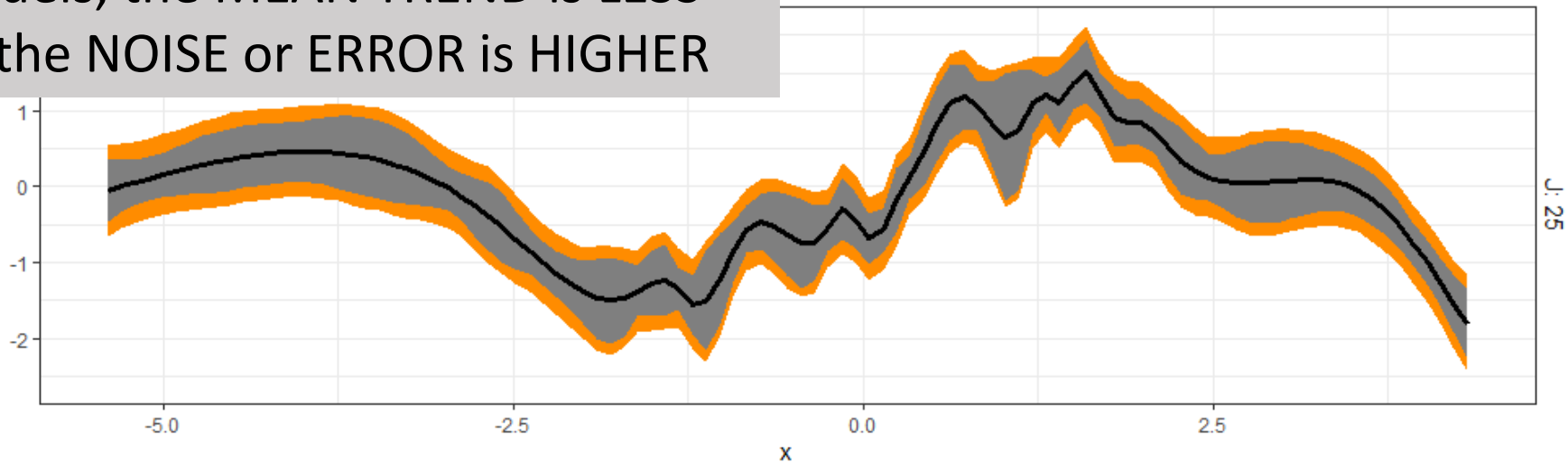


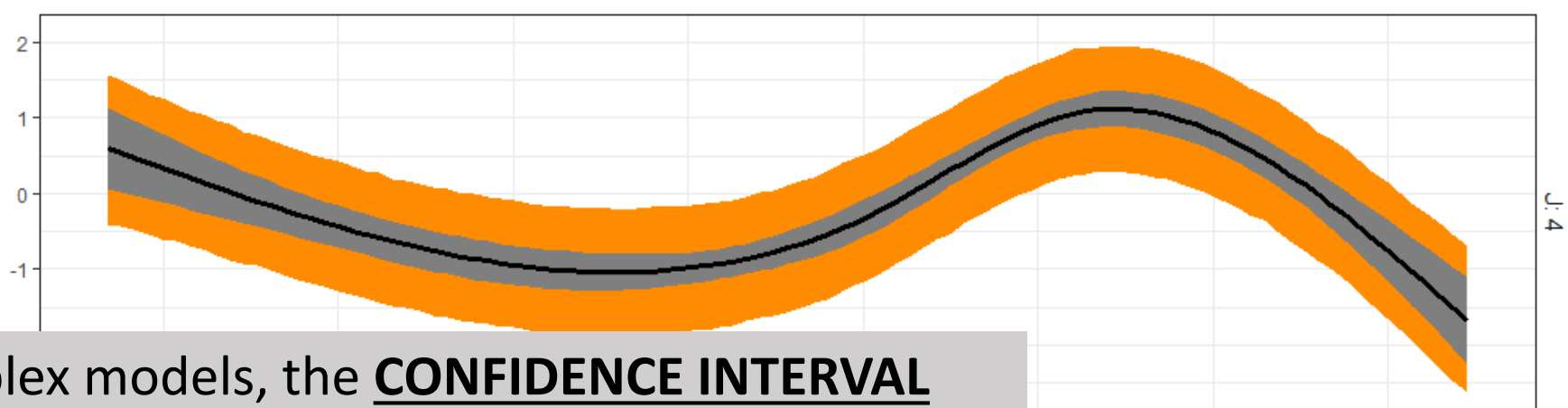


Look at the relationship between the confidence interval and prediction interval

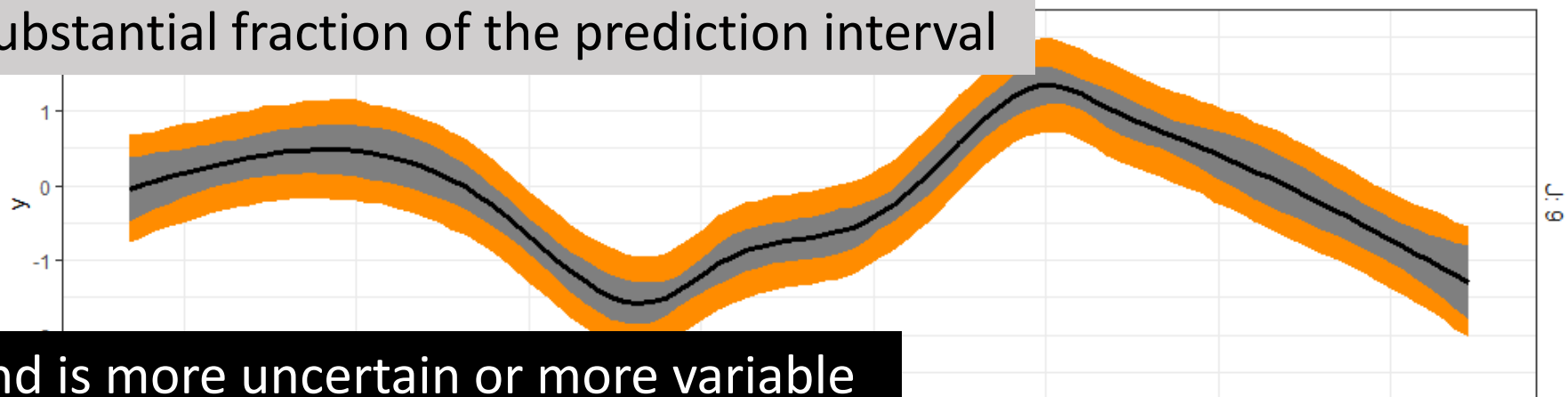


For simpler models, the MEAN TREND is LESS VARIABLE, but the NOISE or ERROR is HIGHER

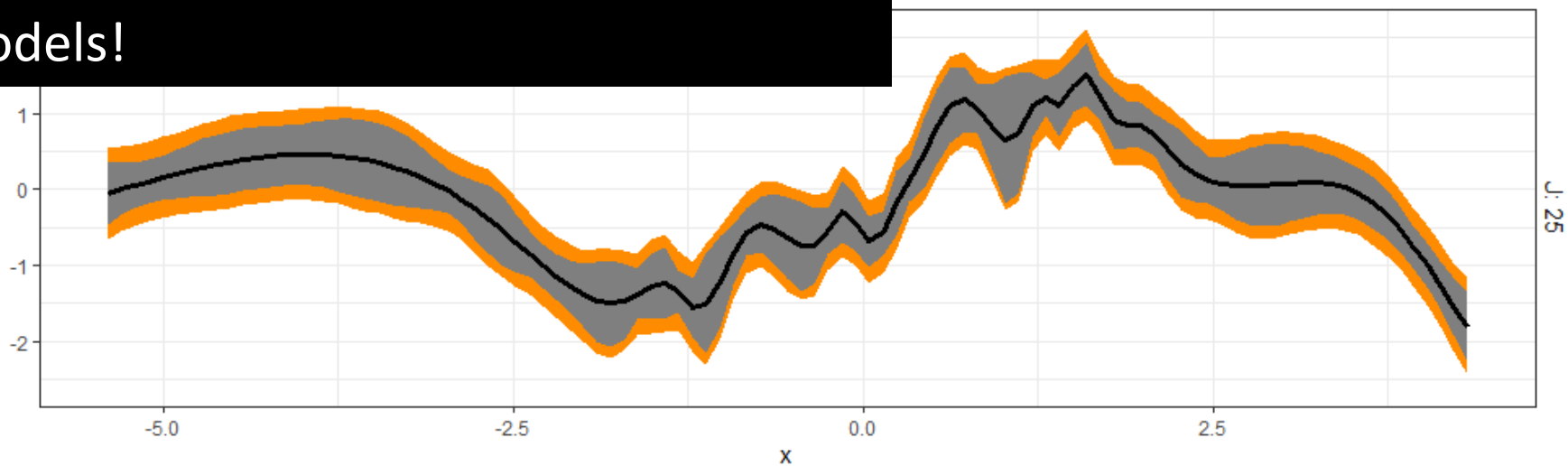




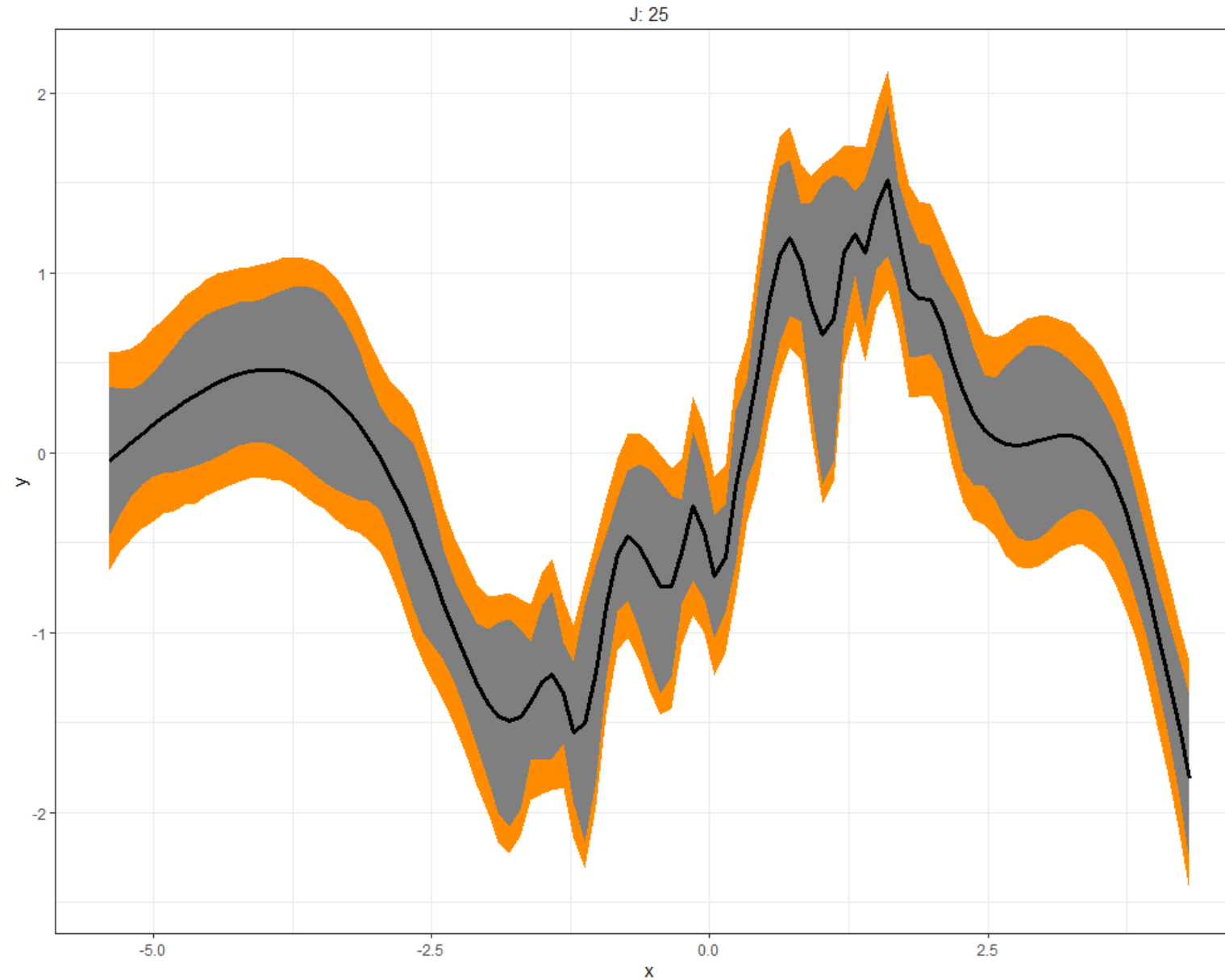
For more complex models, the **CONFIDENCE INTERVAL** “takes up” a substantial fraction of the prediction interval



The mean trend is more uncertain or more variable for complex models!



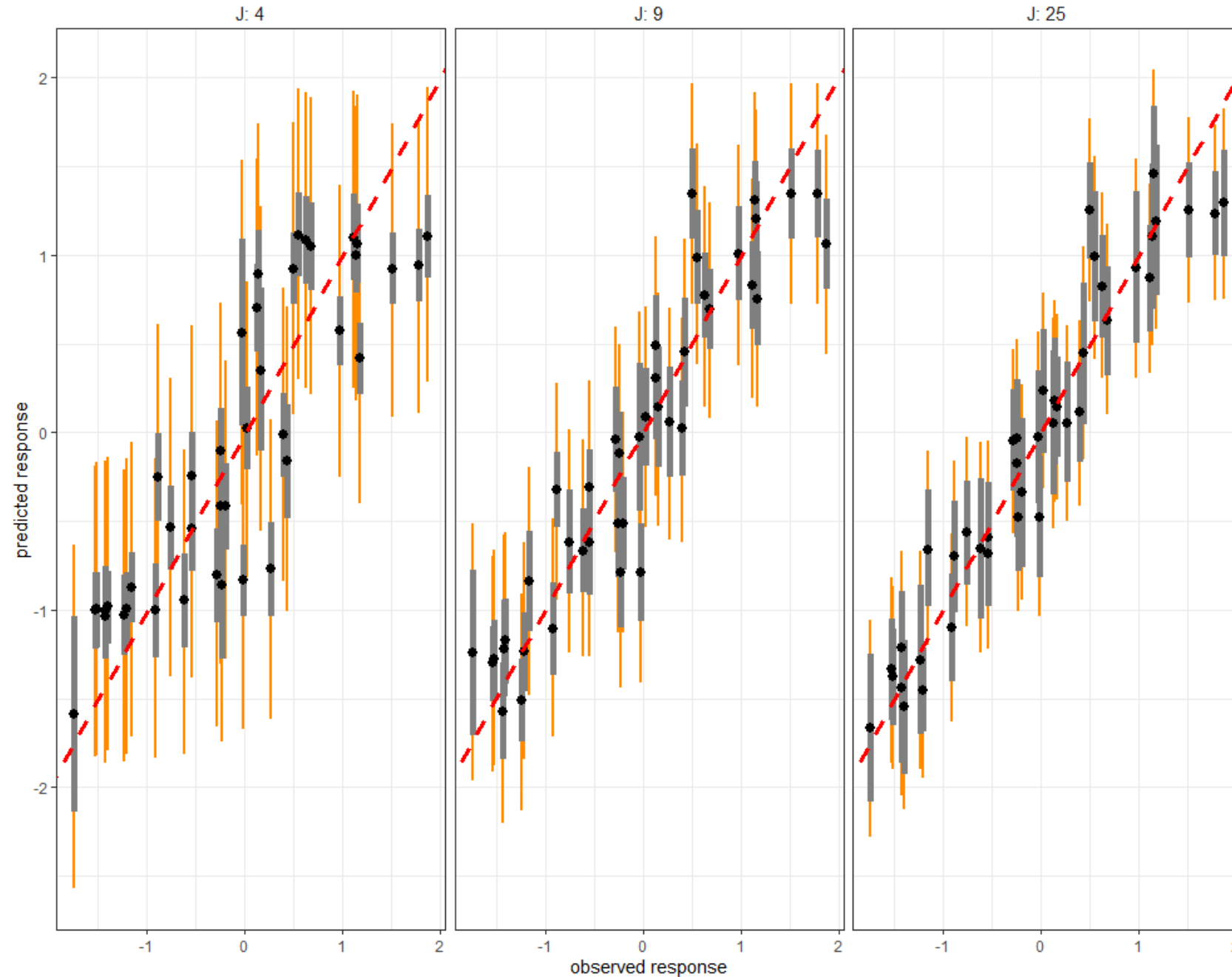
The 25 degree-of-freedom spline alone, to make it easier to see the “wiggles” and the relationship between the confidence interval and prediction interval



Even though the noise, σ , is lower for the most complex model...clearly the 25 DOF spline does not **generalize**

- Why do we know the 25 DOF spline does not generalize?
- Because we know the true function in this toy demo!
- However, if we consider what the fitting procedure “sees”...it’s trying to **drive down the error** between the mean trend and the response, as much as the prior will allow.

Predicted vs observed figure based on the training set for each of the 3 spline models



We have already discussed how assessing model performance on the training set alone can lead to overfitting, in non-Bayesian settings

- **The same goes for Bayesian models!**
- Calculating the RMSE, MAE, and R-squared on the training set would make it seem that the 25 DOF spline is the best.

What's different about performance metrics in a Bayesian model compared to non-Bayesian models?

- UNCERTAINTY on the β parameters induce UNCERTAINTY on the mean trend, μ .
- UNCERTAINTY on the mean trend induces UNCERTAINTY in the performance metrics!
- The UNCERTAINTY however, when assessed ONLY on the training set will NOT prevent overfitting!

Regression performance metrics based on $s = 1, \dots, S$ posterior samples

- Mean Squared Error:

$$MSE_s = \frac{1}{N} (\mathbf{y} - \Phi \boldsymbol{\beta}_s)^T (\mathbf{y} - \Phi \boldsymbol{\beta}_s)$$

- Root Mean Squared Error:

$$RMSE_s = \sqrt{MSE_s}$$

- Mean Absolute Error:

$$MAE_s = \frac{1}{N} \sum_{n=1}^N (|y_n - \boldsymbol{\phi}_{n,:} \boldsymbol{\beta}_s|)$$

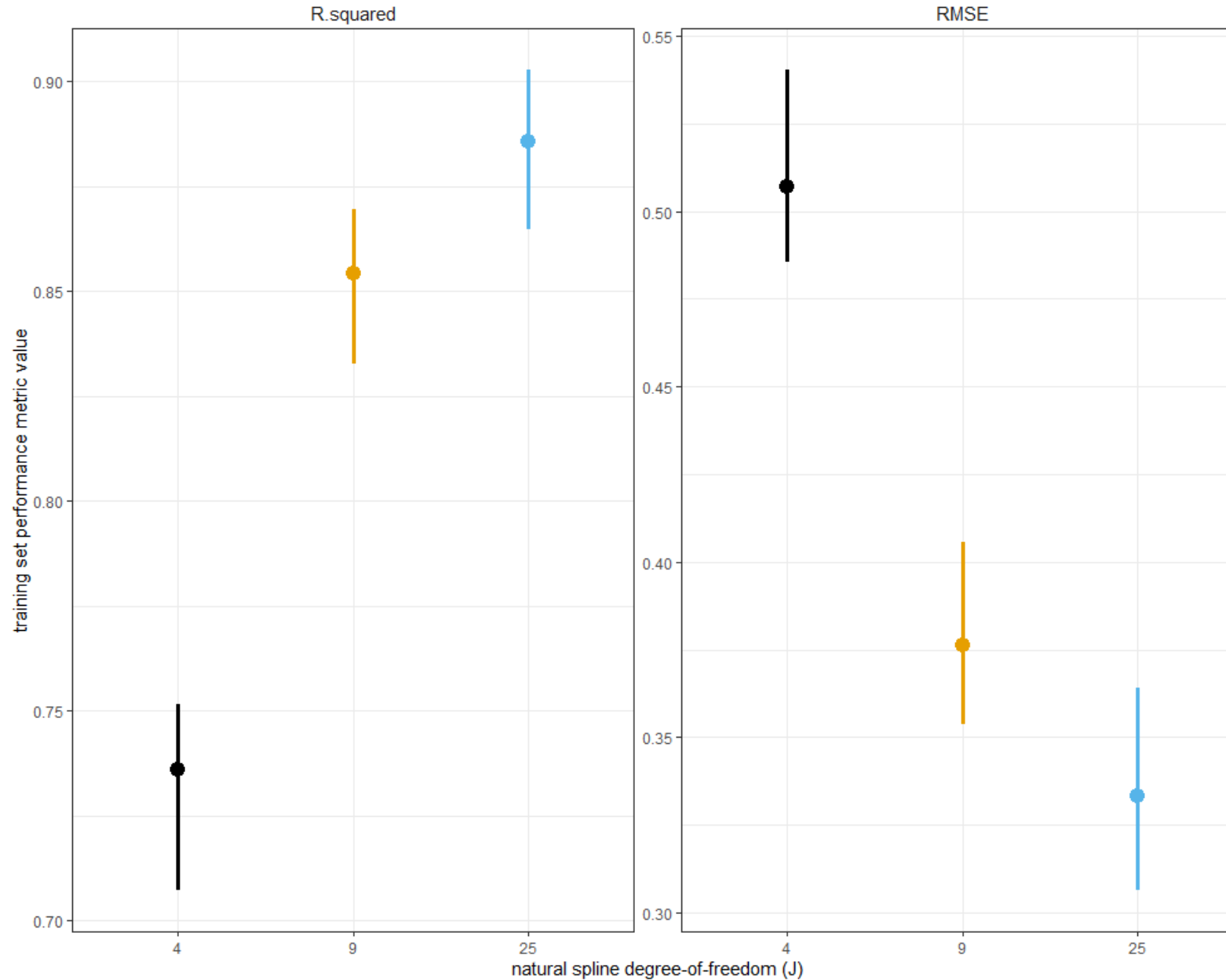
Regression performance metrics based on $s = 1, \dots, S$ posterior samples

- Bayesian R-squared:

$$R_s^2 = \frac{\text{var}(\mu_s)}{\text{var}(\mu_s) + \text{var}(y - \mu_s)}$$

- $\text{var}(\mu_s)$ denotes the variance of the mean trend for the s -th posterior sample across the N observations
- $\text{var}(y - \mu_s)$ denotes the variance of the residual associated with s -th posterior sample across the N observations
- Compare to classical R-squared: $R^2 = 1 - \frac{MSE(\hat{\beta})}{\text{var}(y)}$

Posterior summaries of the R-squared and RMSE performance metrics assessed on the **TRAINING** set



However, let's consider an alternative approach to assessing model performance

- When Bayes' Theorem was introduced it was written as:

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}}$$

So far...we have focused heavily on the numerator...

$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior}$$

- We have ignored the denominator, the Evidence, up to now because it is very difficult to compute.

The evidence is the AVERAGE prior predictive density of the data

- If we denote all unknown parameters as $\boldsymbol{\theta}$, the evidence integrates the likelihood over the prior.

$$\text{Evidence} = p(\mathbf{y} \mid \boldsymbol{\Phi}) = \int p(\mathbf{y} \mid \boldsymbol{\Phi}, \boldsymbol{\theta}) p(\boldsymbol{\theta}) d\boldsymbol{\theta}$$

A linear model of a single input with a linear relationship has 3 unknowns

- The Evidence is a triple integral!

$$\text{Evidence} = p(\mathbf{y} \mid \mathbf{X}) = \iiint p(\mathbf{y} \mid \mathbf{X}, \beta_0, \beta_1, \sigma) p(\beta_0, \beta_1, \sigma) d\beta_0 d\beta_1 d\sigma$$

For our 9 degree of freedom spline...**the Evidence is an integral over 11 variables!**

- For most practical problems we cannot evaluate the Evidence.
- The Evidence is too difficult to calculate!

If we could calculate it, the **Evidence** could be used to evaluate the performance of the model!

- Consider the general linear basis model Evidence:

$$p(\mathbf{y} \mid \mathbf{\Phi}) = \int p(\mathbf{y} \mid \mathbf{\Phi}, \boldsymbol{\theta}) p(\boldsymbol{\theta}) d\boldsymbol{\theta}$$

If we could calculate it, the **Evidence** could be used to evaluate the performance of the model!

- Consider the general linear basis model Evidence:

$$p(\mathbf{y} \mid \Phi) = \int \underbrace{p(\mathbf{y} \mid \Phi, \boldsymbol{\theta})}_{\text{Likelihood}} p(\boldsymbol{\theta}) d\boldsymbol{\theta}$$

Represents how well the
model fits the observations

If we could calculate it, the **Evidence** could be used to evaluate the performance of the model.

- Consider the general linear basis model

Prior

All allowable values of the parameters

$$p(\mathbf{y} \mid \Phi) = \int \underbrace{p(\mathbf{y} \mid \Phi, \boldsymbol{\theta})}_{\text{Likelihood}} \overbrace{p(\boldsymbol{\theta})}^{\text{Prior}} d\boldsymbol{\theta}$$

Likelihood

Represents how well the model fits the observations

If we could calculate it, the **Evidence** could be used to evaluate the performance of the model.

- Consider the general linear basis model

Prior
All allowable values of
the parameters

$$p(\mathbf{y} \mid \Phi) = \int \underbrace{p(\mathbf{y} \mid \Phi, \boldsymbol{\theta})}_{\text{Likelihood}} \overbrace{p(\boldsymbol{\theta})}^{\text{Prior}} d\boldsymbol{\theta}$$

Integrate over ALL parameters in the model

Likelihood
Represents how well the model fits the observations

If we could calculate it, the **Evidence** could be used to evaluate the performance of the model.

- Consider the general linear basis model

Prior
All allowable values of
the parameters

$$p(\mathbf{y} \mid \Phi) = \int \underbrace{p(\mathbf{y} \mid \Phi, \boldsymbol{\theta})}_{\text{Likelihood}} \underbrace{p(\boldsymbol{\theta})}_{\text{Prior}} d\boldsymbol{\theta}$$

Integrate over ALL parameters in the model

Likelihood
Represents how well the model fits the observations

By integrating over the prior, the **Evidence** or **Marginal Likelihood** sees all combinations of the parameters that allow the model to fit the data well **AND** all combinations of parameters that cause the model to perform poorly!!!

Laplace Approximation provides an estimate to the Evidence!

- It is not perfect, but it can be quite useful.
- Define the posterior mode of all P parameters in the model as $\hat{\theta}$.
- The determinant of the Hessian matrix evaluated at the posterior mode will be denoted as $|\mathbf{H}|_{\hat{\theta}}$.

The Laplace Approximation's estimate to the $\log[\text{Evidence}]$ is:

$$\frac{P}{2} \log[2\pi] - \frac{1}{2} \log[|\mathbf{H}|_{\hat{\boldsymbol{\theta}}}] + \log[p(\mathbf{y} \mid \boldsymbol{\Phi}, \hat{\boldsymbol{\theta}})] + \log[p(\hat{\boldsymbol{\theta}})]$$

The Laplace Approximation's estimate to the $\log[\text{Evidence}]$ is:

$$\frac{P}{2} \log[2\pi] - \frac{1}{2} \log[|\mathbf{H}|_{\hat{\boldsymbol{\theta}}}] + \log[p(\mathbf{y} \mid \boldsymbol{\Phi}, \hat{\boldsymbol{\theta}})] + \log[p(\hat{\boldsymbol{\theta}})]$$

Log-likelihood evaluated
at the posterior mode

Increases as the data fit
increases

The Laplace Approximation's estimate to the $\log[\text{Evidence}]$ is:

Increases if the posterior mode is consistent with our prior guess

Log-prior evaluated at the posterior mode

$$\frac{P}{2} \log[2\pi] - \frac{1}{2} \log[|\mathbf{H}|_{\hat{\boldsymbol{\theta}}}] + \log[p(\mathbf{y} \mid \boldsymbol{\Phi}, \hat{\boldsymbol{\theta}})] + \log[p(\hat{\boldsymbol{\theta}})]$$

Log-likelihood evaluated at the posterior mode

Increases as the data fit increases

The Laplace Approximation's estimate to the $\log[\text{Evidence}]$ is:

Acts as a penalty term!!!

Has a negative sign!!!!

Log-determinant of the
Hessian matrix evaluated
at the posterior mode

Increases if the posterior
mode is consistent with
our prior guess

Log-prior evaluated at the
posterior mode

$$\frac{P}{2} \log[2\pi] - \frac{1}{2} \log[|\mathbf{H}|_{\hat{\boldsymbol{\theta}}}] + \log[p(\mathbf{y} \mid \boldsymbol{\Phi}, \hat{\boldsymbol{\theta}})] + \log[p(\hat{\boldsymbol{\theta}})]$$

Log-likelihood evaluated
at the posterior mode

Increases as the data fit
increases

The Laplace Approximation's estimate to the $\log[\text{Evidence}]$ is:

Acts as a penalty term!!!

Has a negative sign!!!!

Log-determinant of the
Hessian matrix evaluated
at the posterior mode

The curvature of the
log-posterior surface
PENALIZES the model
performance!!!

$$\frac{P}{2} \log[2\pi] - \frac{1}{2} \log[|\mathbf{H}|_{\hat{\boldsymbol{\theta}}}] + \log[p(\mathbf{y} \mid \boldsymbol{\Phi}, \hat{\boldsymbol{\theta}})] + \log[p(\hat{\boldsymbol{\theta}})]$$

Log-likelihood evaluated
at the posterior mode

Increases as the data fit
increases

Comparing models based on their Evidence values with Bayes Factors

- Denote the Evidence associated with model a as: $p(\mathbf{y} \mid \mathbf{\Phi}, M_a)$
- The Bayes Factor is the ratio of the Evidence or Marginal Likelihood between two models:

$$BF = \frac{p(\mathbf{y} \mid \mathbf{\Phi}, M_0)}{p(\mathbf{y} \mid \mathbf{\Phi}, M_1)}$$

- If the Bayes Factor is much greater than 1 it is expected that Model 0 is more plausible than Model 1.

Extend the Bayes Factor idea to multiple models...posterior model weights

- The posterior model weight associated with model k is:

$$w_k = \frac{\exp(\log[p(\mathbf{y} \mid \Phi, M_k)])}{\sum_{k'}^K (\exp(\log[p(\mathbf{y} \mid \Phi, M_{k'})]))}$$

The denominator sums the Evidence across all K candidate models.

The posterior model weight is therefore between 0 and 1.

To demonstrate, rather than comparing the 3 models...let's compare 30 different models!

```
### fit many models
manage_spline_fit <- function(J, logpost_func, train_data, my_settings)
{
  X <- model.matrix(~ splines::ns(x, J), train_data)

  my_settings$design_matrix <- X

  init_beta <- rnorm(ncol(X))

  init_phi <- log(rexp(n = 1, rate = my_settings$sigma_rate))

  my_laplace(c(init_beta, init_phi), logpost_func, my_settings)
}

info_list <- list(
  yobs = demo_sine$y,
  mu_beta = 0,
  tau_beta = 5,
  sigma_rate = 1
)

set.seed(1231)
all_spline_models <- purrr::map(1:30, manage_spline_fit,
                               logpost_func = lm_logpost,
                               train_data = demo_sine,
                               my_settings = info_list)

### extract the log-evidence
model_evidence <- purrr::map_dbl(all_spline_models, "log_evidence")
```

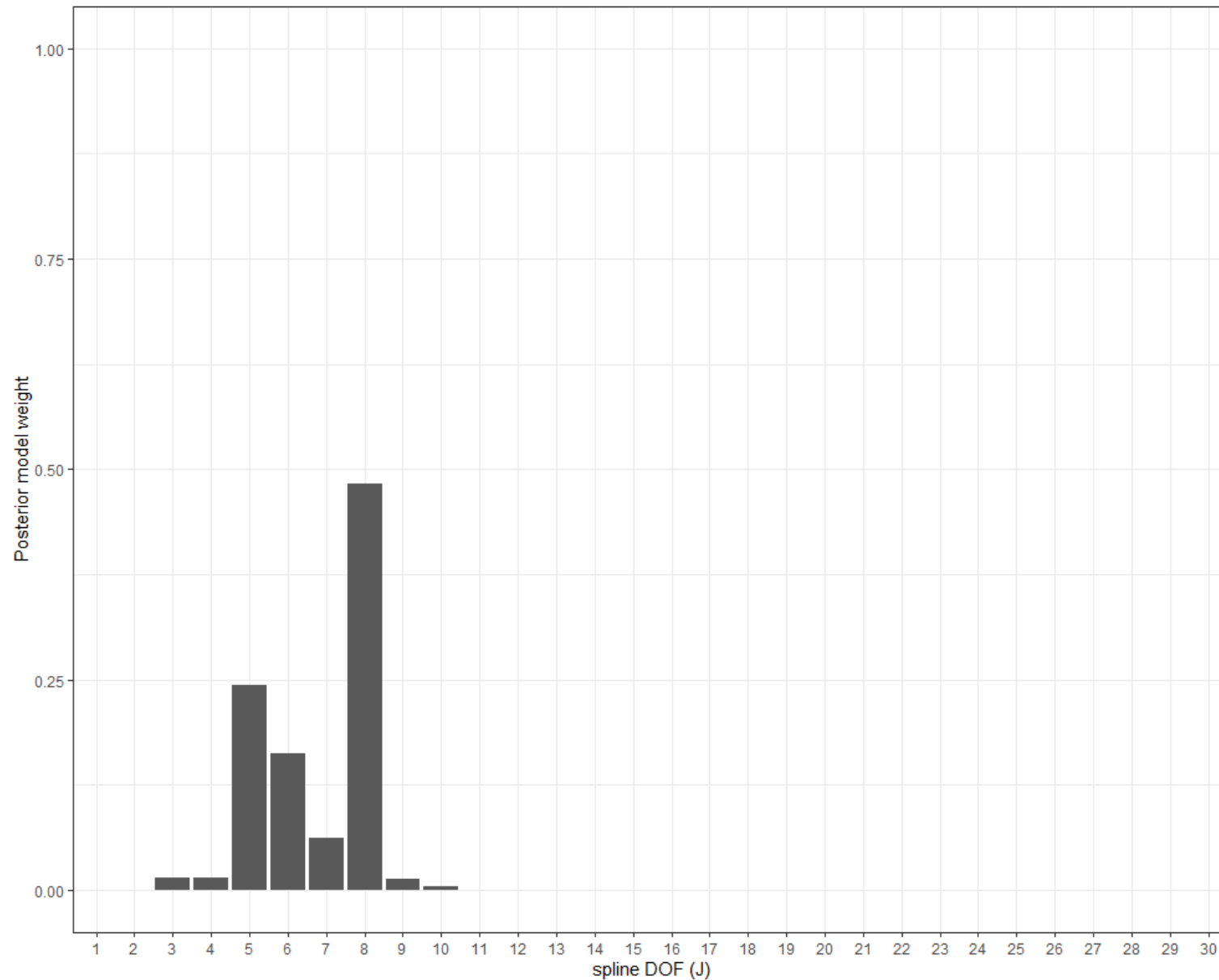
Define a function which allows programmatically setting the spline DOF by the argument J .

Define response and prior hyperparameters

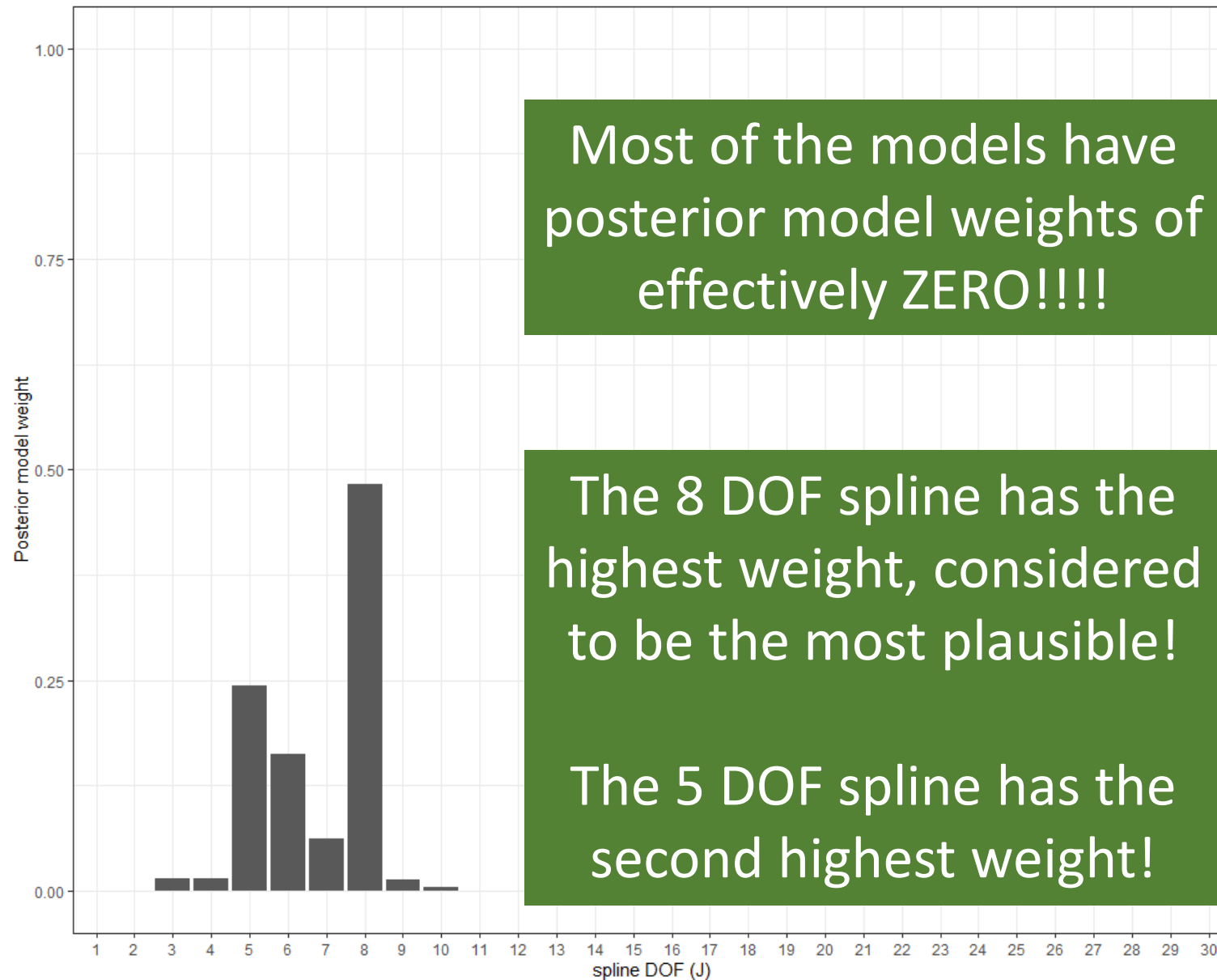
Train up to a 30th DOF spline!

Extract the log-evidence

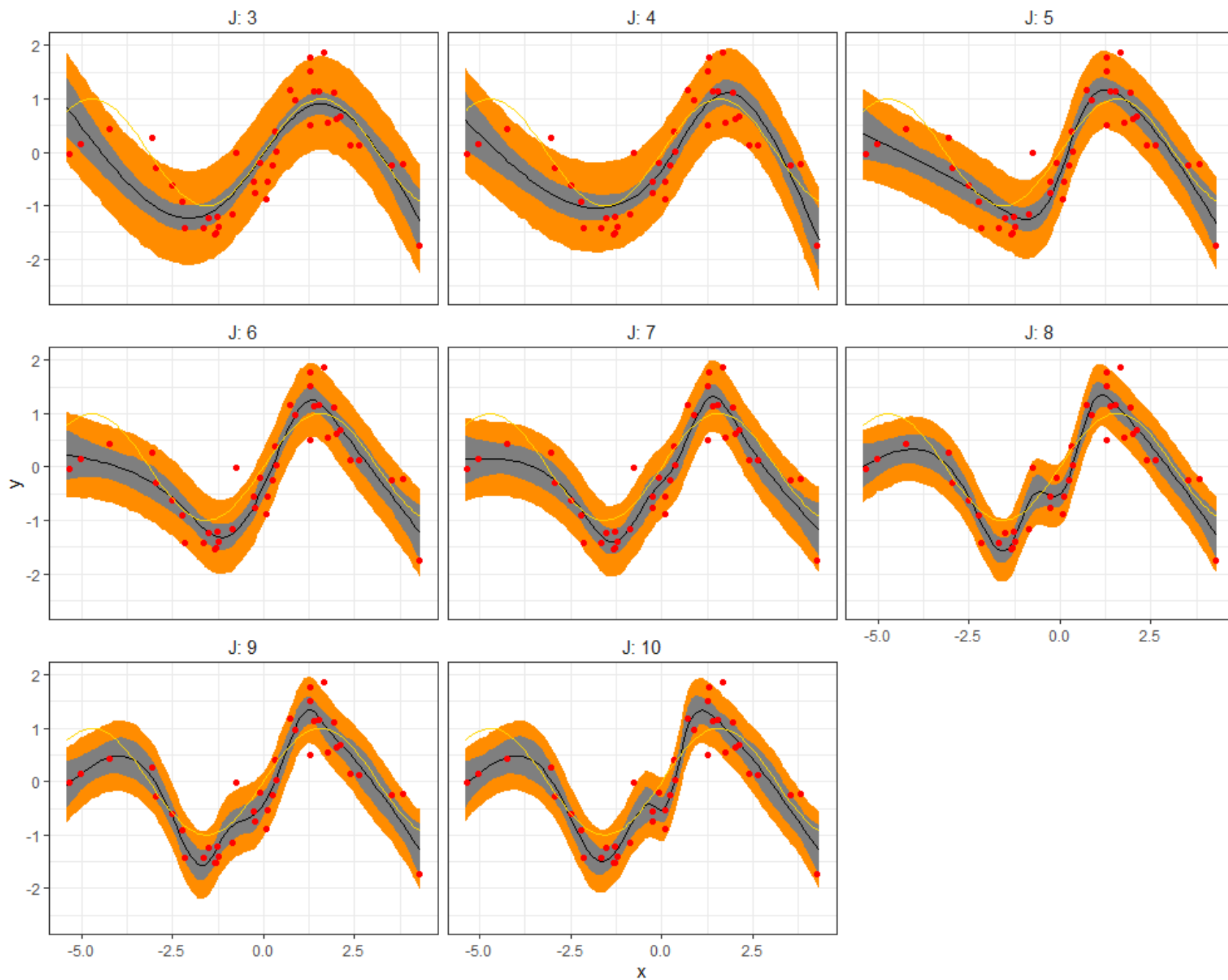
Display the posterior model weights as a bar chart



Display the posterior model weights as a bar chart



Visually compare the predictions of the models with the non-zero model weights



Let's try to gain some insight as to why the 8 DOF model has the highest posterior weight

- The Laplace Approximation's estimate to the $\log[\text{Evidence}]$ is:

$$\frac{P}{2} \log[2\pi] - \frac{1}{2} \log[|\mathbf{H}|_{\hat{\boldsymbol{\theta}}}] + \log[p(\mathbf{y} \mid \boldsymbol{\Phi}, \hat{\boldsymbol{\theta}})] + \log[p(\hat{\boldsymbol{\theta}})]$$

- If we would assume an **infinitely diffuse prior**...the expression simplifies to:

$$\frac{P}{2} \log[2\pi] - \frac{1}{2} \log[|\mathbf{H}|_{\hat{\boldsymbol{\theta}}}] + \log[p(\mathbf{y} \mid \boldsymbol{\Phi}, \hat{\boldsymbol{\theta}})]$$

The log-determinant of the Hessian can be approximated assuming it is well behaved...

- The log-determinant of the hessian matrix requires summing up the local curvature associated with each of the N likelihoods.
- We can approximate this quantity assuming some fixed constant matrix $\mathbf{A}$:

$$\log[|\mathbf{H}|] \approx \log[N^P |\mathbf{A}|] = P \log[N] + \log[|\mathbf{A}|]$$

That seems odd...let's consider a case where the Hessian matrix is DIAGONAL

$$\mathbf{H} = \begin{bmatrix} a_1 & \cdots & 0 \\ \vdots & a_p & \vdots \\ 0 & \cdots & a_P \end{bmatrix}$$

- The determinant of a DIAGONAL matrix is the PRODUCT of the elements along the main diagonal!

$$|\mathbf{H}| = \prod_{p=1}^P (a_p)$$

For a DIAGONAL Hessian...

- The log of the determinant of the Hessian is then just the sum of the log of the main diagonal!

$$\log[|\mathbf{H}|] = \sum_{p=1}^P (\log[a_p])$$

- IF the elements are all equal, $a_p = a, p = 1, \dots, P$, this further simplifies to:

$$\log[|\mathbf{H}|] = \sum_{p=1}^P (\log[a]) = P \times \log[a]$$

For a DIAGONAL Hessian...

- The log of the determinant of the Hessian is then just the sum of the log of the main diagonal!

$$\log[|\mathbf{H}|] = \sum_{p=1}^P (\log[a_p])$$

- IF the elements are all equal, $a_p = a, p = 1, \dots, P$, this further simplifies to:

$$\log[|\mathbf{H}|] = \sum_{p=1}^P (\log[a]) = \boxed{P} \times \log[a]$$

The log-determinant of the Hessian matrix then SCALES as the number of TERMS!!!!!!!

Substitute in this approximation into the expression for the $\log[\text{Evidence}]$

$$\log[p(\mathbf{y} \mid \mathbf{\Phi}, \hat{\boldsymbol{\theta}})] - \frac{1}{2}P \log[N] - \frac{1}{2}\log[|\mathbf{A}|]$$

Drop all terms in the approximation that do not involve the number of observations.

Substitute in this approximation into the expression for the $\log[\text{Evidence}]$

- The Laplace Approximation's estimate to the $\log[\text{Evidence}]$ simplifies to:

$$\log[p(\mathbf{y} \mid \Phi, \hat{\boldsymbol{\theta}})] - \frac{1}{2} P \log[N]$$

The Bayesian Information Criterion (BIC)

- The BIC penalizes the data fit, via the log-likelihood, based on the number of parameters!!!

$$\log[p(\mathbf{y} \mid \Phi, \hat{\boldsymbol{\theta}})] - \frac{1}{2} P \log[N]$$

Complex models with more parameters are penalized.

The additional flexibility offered by having more parameters must be worth it!!!

Even though BIC has Bayesian in the name...it's not actually Bayesian!

- After all, we ignored the influence of the prior...
- However, it is a commonly applied INFORMATION CRITERION because frankly it's very easy to compute.
- It is usually interpreted to represent finding the true model out of a set of candidate models.

Other Information Criterion exist

- Akaike Information Criterion (AIC), the Deviance Information Criterion (DIC), and the Widely Applicable Information Criterion (WAIC)
- All information criterion metrics try and estimate how a model will perform on new data, based only on the behavior on the training set.
- All penalize complexity in some way. The AIC is very similar to the BIC, except it penalizes based on the number of parameters less harshly than the BIC.

AIC and BIC are usually written differentially than presented here...

- Typical formulation uses the negative log-likelihood evaluated at the MLEs...

$$BIC = P \times \log[N] - 2 \times \log[p(\mathbf{y} \mid \Phi, \hat{\theta})]$$

$$AIC = 2P - 2 \times \log[p(\mathbf{y} \mid \Phi, \hat{\theta})]$$

- Interpretations are the SAME though.
- The best model through the above definitions would be the model with the **LOWEST** AIC/BIC value.

In this course, we will use the Laplace Approximation's estimate to the $\log[\text{Evidence}]$

- Deriving the BIC helps give a useful intuitive sense to what the penalty term is doing.
- As more parameters are added to the model, a negative contribution to the Evidence is repeatedly applied.
- Simple models are those that strike a balance in “goodness of fit” with a moderate number of parameters.

Important points about the Information Criterion metrics

- All models MUST be compared on the EXACT same data set.
- Also, if we change the candidate models, the context of a “good value” may also change.
- For example, what would happen if we would include the true sine wave basis function in our candidate list of models?
- Fit the Bayesian model for the sine wave basis and calculate the Laplace Approximation estimate to the $\log[\text{Evidence}]$

All spline models are effectively ruled out!!!!!!

